

SCM-GNN: A Graph Neural Network-Based Multi-Antenna Spectrum Sensing in Cognitive Radio

Youqiang Dong[✉], Min Zhang[✉], *Member, IEEE*, Xi Cheng[✉], *Student Member, IEEE*, and Hai Wang[✉]

Abstract—Spectrum Sensing plays a crucial role in cognitive radio and serves as a fundamental requirement for achieving dynamic spectrum access. This work investigates a novel multi-antenna spectrum sensing framework based on graph neural networks to accurately identify the state of primary users. Specifically, the work proposes a graph spectral convolution-based spectrum sensing scheme (SCM-GNN), which employs stacked graph convolutions to capture the dependencies contained in test statistics. To further enhance the detection performance of SCM-GNN, the work introduces a covariance matrix with smooth factor as the test statistic. The covariance matrix includes more discriminative information and assists the SCM-GNN in achieving state-of-the-art detection performance. Simulation results demonstrate that the proposed algorithm outperforms existing works in terms of detection performance under the influence of various non-ideal factors, such as general Gaussian noise, channel fading, large-scale fading, real-world scenario, and imperfect reporting channel.

Index Terms—Cognitive radio, multi-antenna spectrum sensing, deep learning, graph convolutional networks.

I. INTRODUCTION

WITH the widespread deployment of the fifth-generation wireless communication systems (5G) [1], [2] and the rapid growth of the Internet of Things (IoT) [3], the escalating demand for higher transmission rates and wider transmission bandwidth cannot be overlooked. However, the conventional spectrum allocation strategy only allows the licensed users to operate on pre-assigned fixed spectrum resources. Unlicensed users are prohibited from using these resources to avoid interfering with the communication of primary users. To improve the efficiency of spectrum utilization, cognitive radio (CR) [4] technology has been proposed, aiming to actively identify and efficiently exploit transmission opportunities while minimizing interference to licensed users. Essentially, CR technology enables unlicensed secondary users to opportunistically access licensed spectrum. When a spectrum hole

appears, the secondary users (SUs) immediately occupy the licensed spectrum for communication.

One of the challenges in CR is to locate and utilize the available spectrum resources. Therefore, spectrum sensing is viewed as a fundamental and crucial task in CR, which has drawn significant attention from academia and industry [5]. Since the inception of spectrum sensing, numerous algorithms have been developed. Current spectrum sensing algorithms are classified into three categories: model-based methods, machine learning-based methods, and deep learning-based methods.

For model-based methods, although the estimator-correlator is the optimal detection algorithm, it requires prior information about primary users (PUs). In an unknown environment, the estimator-correlator cannot be implemented and fails to identify the available spectrum due to the lack of statistical information. Therefore, semi-blind detection methods are more suitable for practical sensing scenarios. In semi-blind detection algorithms, the energy detection (ED) algorithm is widely researched due to its simple implementation and low computational complexity [6]. In [7], the authors have proposed a maximum-eigenvalue detection (MED) algorithm to detect highly correlated signals. Zhang et al. [8] have designed a detection method based on the generalized likelihood ratio test (GLRT), which utilizes the eigenvalues of signal subspace to construct the test statistic. Although semi-blind detection algorithms only require noise power for spectrum sensing, the detection performance is easily disturbed by noise uncertainty. To mitigate the impact of noise uncertainty, totally-blind detection algorithms, such as BCED [9], and CAV [10], have been proposed without requiring any prior information. In [11], the authors have utilized the maximum and minimum eigenvalue (MME) of covariance to construct a test statistic, with the detection threshold associated with the Tracy-Widom distribution. Zhao et al. [12] have proposed a universal eigenvalues-based sensing method. Additionally, some totally-blind algorithms based on eigenvalues and geometric features have been proposed, such as ME-AM, ME-GM, and AGM.

Although model-based methods demonstrate great detection performance, a drawback of these methods is that their statistical models are constructed based on theoretical statistical models. In practical scenarios, the discrepancy between actual statistical models and theoretical statistical models results in the degradation of detection performance of conventional algorithms. Due to its drawbacks, machine learning and deep learning-driven algorithms have attracted increasing attention

Manuscript received 27 December 2023; revised 23 April 2024 and 2 July 2024; accepted 10 July 2024. Date of publication 22 July 2024; date of current version 7 February 2025. This work was supported by the National Natural Science Foundation of China (No.12003018). The associate editor coordinating the review of this article and approving it for publication was Y. Sagduyu. (Corresponding author: Hai Wang.)

The authors are with the School of Aerospace Science and Technology, Xidian University, Xi'an 710071, China (e-mail: yqdong_2@stu.xidian.edu.cn; minzhang@xidian.edu.cn; wanghai@mail.xidian.edu.cn).

Digital Object Identifier 10.1109/TCCN.2024.3431923

from scholars. Some studies have employed machine learning (ML) [13] to assist in the identification of spectrum occupancy states. Several SVM-based learning systems [14], [15], [16] have been proposed to identify the state of primary users and determine wireless communication parameters. In [17], the authors have proposed an ANN-based spectrum sensing method that utilizes received signal strength and cyclostationary information as inputs to the ANN. In [18], a Bayesian ML-based method has been proposed to identify global spectrum states. The ML-based method effectively exploits the mobility of secondary users to enhance the temporal and spatial diversity of the sensed signals. In [19], K-means, GMM, and KNN-based algorithms have been proposed for spectrum sensing under NOMA. However, these machine learning-based algorithms have inherent limitations in extracting deep features. When performing spectrum sensing in non-ideal channel environments, these algorithms cannot reliably provide accurate detection results.

Compared to these machine learning-based methods, deep learning-based algorithms are data-driven and can extract deeper features from extensive sensing samples. In contrast to conventional algorithms, deep learning-based algorithms do not require the construction of complex statistical models. Convolutional Neural Networks (CNNs) have achieved significant success in various domains due to their powerful feature extraction capabilities, and have gained attention in spectrum sensing tasks. In [20], the authors have developed a deep CM-CNN algorithm that uses a covariance matrix (CM) as an input for spectrum sensing. The CM-CNN algorithm employs a limited number of convolutional layers, pooling layers, and fully connected layers for spectrum detection. Chen et al. [21] have proposed a novel approach using the short-time Fourier transform (STFT) to construct the time-frequency matrix of received samples as a test statistic. In [22], the authors have developed a detection approach combining a time-frequency matrix based on wavelet transform with a residual network. The approach effectively conducts the spectrum sensing for non-stationary signals at low signal-to-noise ratios. In contrast to the method described in [22], Cheng et al. [23] have proposed a novel environment-robust spectrum sensing method based on a matching network. The approach considers the cross-correlation coefficient of the Cyclic Prefix (CP) of Orthogonal Frequency Division Multiplexing (OFDM) signals as input to the algorithm. In [24], the authors have proposed a data-driven CNN as a suitable alternative solution to the likelihood ratio test (LRT) under various general noise scenarios, including Middleton class A noise, isometric complex symmetric-stable noise, and isometric complex generalized Gaussian noise. These detection algorithms focus on utilizing the CNN framework to extract spatial features from input statistics. Furthermore, researchers have concentrated on capturing temporal features from historical received data. In [25], the authors have proposed a CNN-based approach that fully combines historical and current sensing data to recognize the state of primary users. To enhance detection performance, several algorithms combining CNN and LSTM have been proposed to capture both spatial and temporal features from sensing data [26], [27]. In [26], the authors have designed

a spectrum sensing architecture based on CNN and LSTM, where the CNN model captures energy correlation features and the LSTM model captures temporal features from multiple sensing periods based on the output of the CNN. Ke et al. [27] have employed a mixed model that combines CNN with LSTM to extract time-frequency features from received data in low SNR. In [28], the authors have designed the DS2MA model, which constructs auto-correlation and cross-correlation matrices for multi-antenna spectrum sensing. However, the aforementioned deep learning-based methods cannot perfectly solve all problems, and their detection performance is limited by input statistics, especially in non-ideal environments.

Few related works in the literature have utilized Graph Neural Networks to extract correlation information from neighboring nodes. A spectrum sensing framework [29] based on a graph convolution network (GCN) has been developed, which captures the relationships among the PU signals sensed by multiple SUs distributed across a geographical region. Zhang et al. [30] have designed a SpecKriging detection algorithm for secure cooperative spectrum sensing (CSS). The SpecKriging method, pre-trained using historical sensing data from a few trusted anchor sensors, is employed to evaluate the trustworthiness of non-anchor sensors' data and to incorporate historical sensing data with new data from anchor sensors for model retraining.

To mitigate the limitations of model-based, machine learning-based, and deep learning-based methods, the work introduces a novel test statistic and designs a graph convolution-based method to enhance the detection performance of multi-antenna spectrum sensing. The introduced covariance matrix denotes a more efficient and comprehensive statistic, encompassing discriminative features, such as energy levels, eigenvalues, and correlation features. Furthermore, the covariance matrix with smooth factor exhibits specific temporal relationships among the submatrices. On the other hand, the GNN is effective at processing graph data in non-Euclidean space and capturing hidden correlations between neighboring nodes. Motivated by this, the work employs a graph neural network to model the temporal relations between submatrices within the introduced covariance matrix and to capture the discriminative features embedded in the covariance matrix, thereby enhancing sensing performance. Therefore, we propose a GNN-based scheme for multi-antenna spectrum sensing, which utilizes covariance matrices with smooth factor as the test statistic. The primary contributions of this work are as follows.

- The work proposes a graph neural network-based spectrum sensing (SCM-GNN) algorithm, which employs the covariance matrix as the test statistic. The algorithm utilizes the graph neural network to capture potential correlations between elements in the covariance matrix to improve detection performance.
- The work introduces the covariance matrix with smooth factor as the test statistic for spectrum sensing. Compared to the conventional covariance matrix, the introduced covariance matrix contains more discriminative features, such as auto-covariance matrices, cross-covariance matrices, auto-correlation features, and cross-correlation

features. Furthermore, the temporal relationships among the submatrices in the introduced covariance matrix are advantageous for the graph neural network to model their correlations, thereby enhancing detection performance.

- This paper evaluates the detection performance of the SCM-GNN algorithm through numerical experiments under various detection environments. The simulation results clearly illustrate that the proposed algorithm exhibits superiority in detection performance, scalability, and robustness.

The remainder of this work is structured as follow. Section II introduces the multi-antenna spectrum sensing system. The math fundamentals of graph convolution are discussed in Section III. Section IV elaborates on SCM-GNN architecture for spectrum sensing. The experimental results and analyses are presented in Section V. Finally, Section VI summarizes the work and provides several orientations for future work.

II. MULTI-ANTENNA SPECTRUM SENSING SYSTEM

A. Brief Review of Spectrum Sensing

In this section, the work considers a general Single-Input-Multiple-Output (SIMO) scenario, where the PU utilizes a single transmitting antenna for signal transmission, while the SU employs a sensing system equipped with M receiving antennas to perform spectrum sensing. The spectrum sensing problem can be regarded as a binary hypothesis testing problem:

$$y_m(t) = \begin{cases} \kappa(t)h_m(t)s(t) + w_m(t) & H_1 \\ w_m(t) & H_0 \end{cases} \quad (1)$$

where $m = 1, 2, \dots, M$, $t = 1, 2, \dots, N$. The hypothesis H_1 and hypothesis H_0 correspond to the presence and absence of the PU, respectively. In equation (1), $s(t)$ and $w_m(t)$ represent the transmission signal and noise, respectively. $h_m(t)$ denotes the channel gain of the m -th antenna. $\kappa(t)$ denotes path loss and shadow fading. N is the length of the observation sample.

B. Non-Ideal Factors

To model the dynamic changes in sensing scenarios, this work considers several non-ideal factors that impact the detection performance of algorithms.

1) *Channel Fading*: The Rayleigh distribution is considered appropriate for describing modern wireless propagation channels in heavily built-up urban environments. It is a continuous probability distribution for non-negative random variables and represents a special case of the Weibull distribution. The probability density function (PDF) [31] is expressed as

$$f_R(x; \varepsilon) = \frac{x}{\varepsilon^2} \exp\left(-\frac{x^2}{2\varepsilon^2}\right) \quad (2)$$

where ε represents the scale parameter, which determines the shape and scale size of the distribution. The Nakagami-m fading channel is widely used to describe the dynamic characteristics of modern wireless fading environments [32]. The PDF of in-phase component $f_{NI}(x)$ and quadrature

component $f_{NQ}(x)$ for complex Nakagami-m distribution can be expressed as [33]

$$\begin{aligned} f_{NI}(x_I; a, p, \Omega) &= \left(\frac{a}{\Omega}\right)^{\frac{a(1+p)}{2}} \frac{|x_I|^{a(1+p)-1}}{\Gamma\left(\frac{a(1+p)}{2}\right)} \exp\left(-\frac{ax_I^2}{\Omega}\right) \\ f_{NQ}(x_Q; a, p, \Omega) &= \left(\frac{a}{\Omega}\right)^{\frac{a(1-p)}{2}} \frac{|x_Q|^{a(1-p)-1}}{\Gamma\left(\frac{a(1-p)}{2}\right)} \exp\left(-\frac{ax_Q^2}{\Omega}\right) \end{aligned} \quad (3)$$

where a and Ω represent the fading parameter and power parameter, respectively. The phase parameter p is utilized to quantify the power offset between the in-phase component x_I and the quadrature component x_Q .

2) *Noise*: The received signals inevitably incorporate noise due to thermal noise and inherent noise in the communication system itself. Additive white Gaussian noise (AWGN) is commonly encountered in communication systems. Additionally, the work employs complex generalized Gaussian noise (CGGN) to simulate pulse noise. The PDF of complex generalized Gaussian distribution (CGGD) is denoted as [34]

$$f_{CGGD}(x; v, \varepsilon, e) = \frac{v}{\pi \varepsilon \Gamma(1/v)} \exp\left(-\left|\frac{x-e}{\varepsilon}\right|^{2v}\right) \quad (4)$$

where the shape, scale, and location parameters of CGGN are denoted as v , ε , and e , respectively. The gamma distribution is represented by Γ .

3) *Large-Scale Fading*: In practical scenarios, large-scale fading caused by mobile devices traversing considerable distances is inevitable. Generally, large-scale fading consists of path loss and shadow fading. The expression of large-scale fading is given by [35]

$$\kappa(n; P, \beta, \alpha, d, h) = \sqrt{\frac{P}{\beta(d(n))^\alpha 10^{\frac{h(n)}{10}}}} \quad (5)$$

where P , β , and α are transmitter power, path loss exponent, and path loss constant, respectively. The distance between the PUs and SUs is denoted by d . The symbol $h(n)$ denotes shadow fading at the time n , which follows a normal distribution with zero mean and variance σ^2 . The PDF of shadow fading is expressed as $f_h(x) = (1/(\sqrt{2\pi}\sigma)) \cdot \exp(-x^2/2\sigma^2)$.

4) *Noise Uncertainty*: Noise uncertainty is an additional non-ideal factor that impacts detection performance in semi-blind spectrum sensing algorithms based on noise variance. Due to the noise uncertainty, the estimated noise power varies randomly within the range of $[\sigma_L^2, \sigma_H^2]$. The noise uncertainty coefficient is denoted as

$$u = 10 \log_{10}(\sigma_H^2 / \sigma_L^2) \quad (6)$$

where σ_H^2 and σ_L^2 indicate the greatest and least noise variance, respectively.

III. MATH FUNDAMENTALS OF GRAPH CONVOLUTION

This section primarily describes the construction of graph data and the fundamentals of graph spectral convolution.

A. Test Statistic Construction

The test statistic is the foundation of graph construction, which determines the effectiveness of the graph construction. The test statistic employed in this work is the covariance matrix with smooth factor, which is defined as

$$R_y(N) = \frac{1}{N} \sum_{t=\zeta-1}^{\zeta-2+N} \tilde{y}(t) \tilde{y}(t)^* \quad (7)$$

$$\tilde{y}(t) = [y^T(t), y^T(t-1), \dots, y^T(t-\zeta+1)]^T$$

$$y(t) = [y_1(t), y_2(t), \dots, y_M(t)]^T$$

where ζ represents the smooth factor, and the symbol $*$ denotes the conjugate transpose. The covariance matrix $R_y(N)$ is of dimensions $\zeta M \times \zeta M$, and $\tilde{y}(t)$ is a vector with dimensions $\zeta M \times 1$. The vector $y(t)$ represents the received signal of the antenna array at time t and is of dimensions $M \times 1$. The elements of $R_y(N)$ correspond to auto-correlation coefficients or cross-correlation coefficients and can be represented as

$$R_y(N, i, j, \tau_1, \tau_2) = \frac{1}{N} \sum_{t=0}^{N-1} y_i(t - \tau_1) \cdot y_j(t - \tau_2)^* \quad (8)$$

$$i, j = 1, 2, \dots, M; \tau_1, \tau_2 = 0, 1, \dots, \zeta - 1$$

where τ_1 and τ_2 represent the sampling delays. When $\tau_1 = \tau_2$ and $i = j$, the $R_y(N, i, j, \tau_1, \tau_2)$ correspond to the diagonal elements of covariance matrix $R_y(N)$ and represent the auto-correlation coefficients. When $\tau_1 \neq \tau_2$ and $i \neq j$, the $R_y(N, i, j, \tau_1, \tau_2)$ correspond to the non-diagonal elements of $R_y(N)$ and denote the cross-correlation coefficients. Furthermore, the time delay covariance matrix is defined as

$$\hat{R}_y(N, \tau_1, \tau_2) = \frac{1}{N} \sum_{t=1}^N y(t - \tau_1) y(t - \tau_2)^* \quad (9)$$

$$y(t) = [y_1(t), y_2(t), \dots, y_M(t)]^T$$

$$\tau_1 = 0, 1, \dots, \zeta - 1, \tau_2 = 0, 1, \dots, \zeta - 1$$

The time delay covariance matrix $\hat{R}_y(N, \tau_1, \tau_2)$ is of dimensions $M \times M$. When both τ_1 and τ_2 are set to 0, the delay covariance matrix is equal to the auto-covariance matrix, i.e., $\hat{R}_y(N) = E(y(t)y(t)^*)$. When $\tau_1 = \tau_2$, the time delay covariance matrix is equivalent to the block diagonal matrix within $R_y(N)$. When $\tau_1 \neq \tau_2$, the time delay covariance matrix, which represents a cross-covariance matrix, can be interpreted as a non-block diagonal matrix within $R_y(N)$. According to the definitions in equation (7) and (9), the time delay covariance matrix is a subset of $R_y(N)$. Therefore, the covariance matrix $R_y(N)$ can be rewritten as

$$R_y(N) = \begin{bmatrix} \hat{R}_y(N, 0, 0) & \dots & \hat{R}_y(N, 0, \zeta - 1) \\ \vdots & \ddots & \vdots \\ \hat{R}_y(N, \zeta - 1, 0) & \dots & \hat{R}_y(N, \zeta - 1, \zeta - 1) \end{bmatrix} \quad (10)$$

The calculation of $R_y(N)$, as demonstrated in equation (7), can be primarily divided into three steps: a). sampling the received data using a rectangular window function with a window amplitude of 1, a window size of (M, ζ) , and a window step of ζ ; b). flattening the sampled data; c).

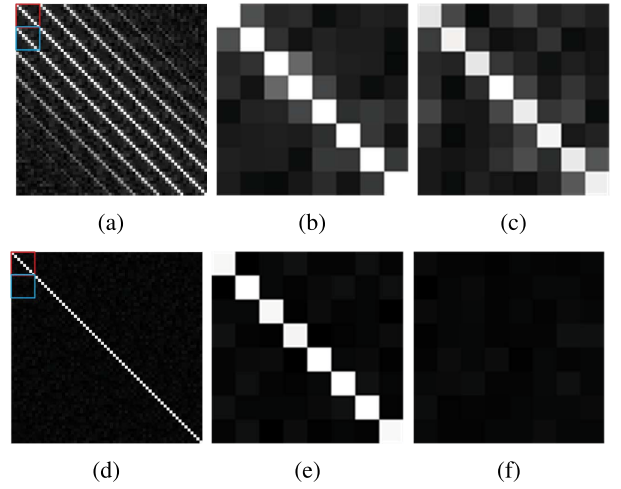


Fig. 1. The grayscale images of covariance matrices under H_1 and H_0 .

calculating its auto-covariance matrix. Therefore, the primary role of the smooth factor is to control the size and step of the rectangular window function. When the smooth factor is set to 1, the covariance matrix with smooth factor degenerates into an auto-covariance matrix. The advantages of using the introduced covariance matrix as the test statistic lie in two aspects. On the one hand, compared to the auto-covariance matrix, the covariance matrix with smooth factor includes more discriminative features, such as the auto-covariance matrices, cross-covariance matrices, auto-correlation features, and cross-correlation-features. Consequently, it enhances the descriptive ability of the test statistic. On the other hand, as shown in Fig. 1, the visual difference between the covariance matrices with smooth factor under H_1 and H_0 is more pronounced than that of the auto-covariance matrix. The difference effectively enhances the ability of GNN to identify the spectrum occupancy state. Fig. 1 shows the grayscale images of covariance matrices under Rayleigh fading. Fig. 1(a) and Fig. 1(d) represent the covariance matrix with smooth factor under H_1 and H_0 , respectively. Fig. 1(b) and Fig. 1(e) denote the auto covariance matrix under H_1 and H_0 , respectively. Fig. 1(c) and Fig. 1(f) represent the time delay covariance matrix with $\tau_1 = 1$ and $\tau_2 = 0$ under H_1 and H_0 , respectively. Since the time delay covariance matrix is a subset of the covariance matrix with smooth factor, Fig. 1(b) and Fig. 1(c) are considered as enlarged versions of the contents within the red and blue boxes in Fig. 1(a), respectively. Fig. 1(e) and Fig. 1(f) also can be considered as enlarged versions of the contents within the red and blue boxes in Fig. 1(d), respectively.

B. Graph Signal Construction

The graph is a universal nonlinear data structure used to represent and store connection relationships between nodes. In this work, an undirected graph is denoted as $\mathcal{G} = (V, X, A, E)$, where V represents a collection of nodes, X denotes node feature matrix, A corresponds to an adjacency matrix, and E denotes a collection of edges. When node $v_i \in$

V points to node $v_j \in V$, the relationship between nodes can be expressed as $e_{ij} = (v_i, v_j) \in E$.

1) *Construction of Adjacency Matrix*: The adjacency matrix is a compact representation to describe the connected relationships between nodes. The attribute of the edge e_{ij} corresponds to the element A_{ij} in the adjacency matrix A and can be measured by cosine similarity.

$$A_{i,j} = \frac{X_i \cdot X_j}{\|X_i\| \|X_j\|} = \frac{\sum_{n=0}^{N-1} X_i(n) \cdot X_j(n)}{\sqrt{\sum_{n=0}^{N-1} X_i(n)^2} \cdot \sqrt{\sum_{n=0}^{N-1} X_j(n)^2}} \quad (11)$$

where $i, j = 1, 2, \dots, Q$ and $i \neq j$. The X_i and X_j denote the node feature of node v_i and node v_j respectively.

2) *Construction of Node Feature*: The attribute of the node v_i corresponds to the node feature X_i , which refers to the characteristic or information associated with the node. In this work, node features are constructed by performing patch embedding operation on the test statistic. The patch embedding operation was originally proposed by Vision Transformer (ViT) [36] to transform images into sequences. Its implementation is exceptionally convenient, requiring only a single convolutional layer. To be more specific, the patch embedding layer utilizes a single convolutional layer with a kernel size of (N_{Patch}, N_{Patch}) and a stride size of S_{Patch} to extract features from the covariance matrix. Then, the output $Z_{Patch} \in \mathcal{R}^{H \times W \times F}$ of the patch embedding is reshaped into a sequence $Z_{Flat} \in \mathcal{R}^{(H \cdot W) \times F}$ of flattened 2D architecture, where (H, W) denotes the size of patch embedding output features, and F represents the number of output channels. Finally, the node features X consists of the flattened output Z_{Flat} of patch embedding, where $X = [X_1; X_2; X_3; \dots; X_Q]$, and $Q = H \times W$ denotes the total number of nodes. Each row vector $Z_{Flat,i} \in \mathcal{R}^{1 \times F}$ of the flattened output corresponds to a node feature X_i , where $X_i = Z_{Flat,i}$.

C. Graph Convolution

In general, graph convolution can be classified into two fundamental methods: spectral-based and spatial-based method. Graph spectral convolution, a fundamental operation in graph neural networks, extracts features of graph signals from the spectral domain perspective. Graph spectral convolution on graph signals involves the following operations: 1). computing graph Fourier transform: perform a Fourier transform on node features X to obtain the spectral representation $\hat{X}$, where $\hat{X} = U^T X$; 2). performing convolution in the spectral domain: perform convolution by multiplying the spectral representation $\hat{X}$ with a filter $\hat{g}_\theta$ to obtain the convoluted node features $\hat{Z}$, where $\hat{Z} = \hat{g}_\theta \hat{X}$; 3). inverting graph Fourier transform: apply the inverse graph Fourier transform on $\hat{Z}$ to obtain the graph-convoluted node features Z . Therefore, the graph spectral convolution on a graph signal can be formulated as

$$Z = U g_\theta(\Lambda) U^T X \quad (12)$$

where node features $X \in \mathcal{R}^{Q \times F}$ and an adjacency matrix $A \in \mathcal{R}^{Q \times Q}$ are taken as input of the graph spectral convolution, and $X_i \in \mathcal{R}^{1 \times F}$ denotes the feature vector of the i -th node

in the graph signal. U and Λ denote the eigenvectors and eigenvalues of the normalized Laplacian matrix L , respectively. The Laplacian matrix L can be written as $L = I_Q - D^{-1/2} A D^{-1/2}$, where $D \in \mathcal{R}^{Q \times Q}$ is the degree matrix with $D_{ii} = \sum_j A_{ij}$. The function $g_\theta(\Lambda)$, which is a non-parametric spectral filter with unconstrained parameters, can be represented as a polynomial filter $g_\theta(\Lambda) = \sum_{k=0}^{K-1} \theta_k \Lambda^k$. K represents the order of polynomials. Therefore, the spectral graph convolution can be rewritten as

$$\begin{aligned} Z &= \sum_{k=0}^{K-1} U \theta_k \Lambda^k U^T X = \sum_{k=0}^{K-1} \theta_k (U \Lambda U^T)^k X \\ &= \sum_{k=0}^{K-1} \theta_k (L)^k X \end{aligned} \quad (13)$$

To circumvent the decomposition of the Laplacian matrix L and decrease computational overhead, Defferrard et al. [37] proposed ChebNet, which is a fast spectral convolution that employs Chebyshev polynomials to approximate the polynomial filter $g_\theta(\Lambda)$. Therefore, ChebNet can be represented as

$$\begin{aligned} Z &= \sum_{k=0}^{K-1} \theta_k P_k(L) X \\ P_k(L) &= 2LP_{k-1}(L) - P_{k-2}(L) \\ P_0 &= 1, P_1 = L \end{aligned} \quad (14)$$

The feature extraction capability of ChebNet is inferior, primarily due to the utilization of illegal coefficients learned by approximating the filter $g_\theta(\Lambda)$. Due to overfitting phenomenon in ChebNet, He et al. [38] proposed ChebNet II, which approximates θ_k by utilizing learnable parameters $\hat{\theta}_j$ and Chebyshev polynomials $P_k(\kappa_j)$. Compared to ChebNet, ChebNet II modifies the filter coefficients by employing Chebyshev interpolation. Therefore, the graph convolution of ChebNet II can be expressed as

$$\begin{aligned} Z &= \sum_{k=0}^{K-1} \theta_k P_k(L) X \\ \theta_k &= \frac{2}{K+1} \sum_{j=0}^K \hat{\theta}_j P_k(\kappa_j) \end{aligned} \quad (15)$$

where $\kappa_j = \cos((j + 0.5)\pi/(K + 1))$.

IV. GRAPH NEURAL NETWORK-BASED SPECTRUM SENSING FRAMEWORK

In this section, the physical meaning of graph signals and the architecture of the SCM-GNN method will be expounded in detail. The constructed graph signals are fed into the graph neural network, which aggregates information from adjacent nodes to construct the decision statistic. The detection threshold is obtained by Monte Carlo method.

A. Graph Signal Construction

A graph signal is composed of node features X and an adjacency matrix A , as discussed in Section III-B2. The node

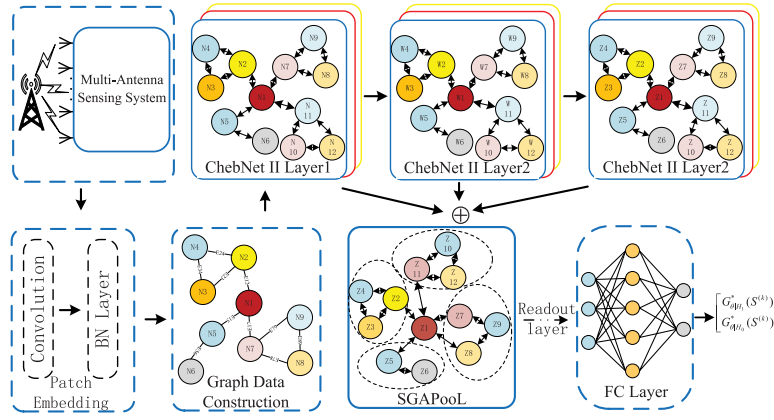


Fig. 2. The framework of the proposed SCM-GNN algorithm.

features X and the adjacency matrix A are of dimensions $Q \times F$ and $Q \times Q$, respectively, where $Q = H \cdot W = ((M \cdot \zeta - N_{Patch})/S_{Patch} + 1)^2$. The physical meanings of the node features and the elements in the adjacency matrix are described in this section. The node features consist of the flattened output features from the patch embedding. Each row vector of the flattened features represents a graph node feature. Therefore, the physical meanings of graph nodes correspond to the receptive fields within a single convolution layer of the patch embedding. The node features are derived from the convolutional mapping of input features within the receptive fields. Specifically, when the covariance matrix with smooth factor is defined as in equation (10), the physical meaning of each node corresponds to a particular time delay covariance matrix. The node feature X_i corresponds to the output feature extracted by a convolutional layer of the patch embedding operation in the receptive fields. For example, when both N_{Patch} and S_{Patch} are set to M , each graph node represents a time delay covariance matrix as defined in equation (9), i.e., $X_1 = \hat{R}_y(N, 0, 0)$, $X_2 = \hat{R}_y(N, 0, 1)$, $\dots$, $X_{\zeta-1} = \hat{R}_y(N, 0, \zeta-1)$, $X_\zeta = \hat{R}_y(N, 1, 0)$, $\dots$, $X_{2\zeta-1} = \hat{R}_y(N, 1, \zeta-1)$, $\dots$, $X_{(\zeta-1) \cdot \zeta} = \hat{R}_y(N, \zeta-1, 1)$, $X_{\zeta^2} = \hat{R}_y(N, \zeta-1, \zeta-1)$. When N_{Patch} is set to $2M$ and S_{Patch} is set to M , each graph node denotes a special covariance matrix composed of multiple time delay covariance matrices, i.e., $X_1 = [\hat{R}_y(N, 0, 0), \hat{R}_y(N, 0, 1); \hat{R}_y(N, 1, 0), \hat{R}_y(N, 1, 1)]$, $X_2 = [\hat{R}_y(N, 0, 1), \hat{R}_y(N, 0, 2); \hat{R}_y(N, 1, 2), \hat{R}_y(N, 1, 2)]$, $\dots$, $X_\zeta = [\hat{R}_y(N, 1, 0), \hat{R}_y(N, 2, 0); \hat{R}_y(N, 1, 1), \hat{R}_y(N, 2, 1)]$, $\dots$, $X_{2\zeta-1} = [\hat{R}_y(N, 1, \zeta-2), \hat{R}_y(N, 1, \zeta-1); \hat{R}_y(N, 2, \zeta-2), \hat{R}_y(N, 2, \zeta-1)]$, $\dots$, $X_{(\zeta-2) \cdot (\zeta-1)} = [\hat{R}_y(N, \zeta-2, 0), \hat{R}_y(N, \zeta-2, 1); \hat{R}_y(N, \zeta-1, 0), \hat{R}_y(N, \zeta-1, 1)]$, $X_{(\zeta-1) \cdot (\zeta-1)} = [\hat{R}_y(N, \zeta-2, \zeta-2), \hat{R}_y(N, \zeta-2, \zeta-1); \hat{R}_y(N, \zeta-1, \zeta-2), \hat{R}_y(N, \zeta-1, \zeta-1)]$. Each edge represents a connected relationship between two nodes, and each element of the adjacent matrix A denotes the similarity between two node features.

B. Graph Neural Network Architecture

Fig. 2 shows the SCM-GNN framework, which includes a convolutional layer (Convolution), a batch normalization

layer (BN), multiple graph convolutional layers (ChebNet II), a graph pooling layer (SAGPool), and two fully connected layers (FC). There are two reasons for constructing the GNN architecture. From the perspective of the elements in the covariance matrix, each element is interpreted as an auto-correlation or cross-correlation coefficient. The GNN can effectively model various relationships among these coefficients, including correlation strength, lag relationship, and periodicity. From the perspective of the patch in the covariance matrix, according to equation (10), the GNN can capture temporal relationships among block diagonal matrices, among non-block diagonal matrices, and between block diagonal and non-block diagonal matrices. The graph neural network architecture is described as follows:

1) *Graph Convolutional Layer*: In general, conventional convolution is designed to operate on regular, local grid structured data. In contrast, graph convolution is a particular convolution that can directly act on non-regular, unstructured graph signals. The output of a graph convolutional layer is a new representation of node features, obtained by a message passing mechanism. The graph convolutional layer in ChebNet II is represented as

$$Z_C^{(\ell+1)} = \sigma \left(\frac{2}{K+1} \sum_{k=0}^K \sum_{j=0}^K \hat{\theta}_j P_k(\kappa_j) P_k(L) f(Z_C^{(\ell)}) \right) \quad (16)$$

where $Z_C^{(\ell+1)} \in \mathcal{R}^{Q \times F_C^{(\ell+1)}}$ and $Z_C^{(\ell)} \in \mathcal{R}^{Q \times F_C^{(\ell)}}$ denote the output and input features of the ℓ -th graph convolutional layer, respectively. $F_C^{(\ell)}$ denotes the dimension of the input features of the ℓ -th graph convolution layer. When $\ell=0$, the input feature matrix $Z_C^{(\ell)}$ is equal to $X \in \mathcal{R}^{Q \times F}$. $f(\cdot)$ and σ denote the MLP function and ReLU activation function, respectively. According to equation (16), the graph convolutional operation involves three steps: a) feeding the input node features $Z_C^{(\ell)}$ into an MLP function to generate new node features; b) generating Chebyshev coefficients θ_k by performing Chebyshev approximation $\sum_{j=0}^K \hat{\theta}_j P_k(\kappa_j)$; c) utilizing Chebyshev polynomials $\frac{2}{K+1} \sum_{k=0}^K \theta_k P_k(L)$ to perform graph convolution.

2) *Graph Pooling Layer*: The work employs the Self-Attention Graph Pooling (SAGPool) [39] as a global graph

pooling layer for pooling operations on graph signals. Firstly, self-attention scores are calculated using a graph convolution layer and an activation function. Based on these self-attention scores, the node selection method in gPool [40] is utilized to remove a portion of nodes from the input graph signal. Then, a mask matrix constructed from the retained nodes is used to perform graph pooling.

Self-attention scores: The self-attention scores $\mathcal{D}_P^{(\ell)}$ are calculated by a graph convolutional layer and are expressed as

$$\mathcal{D}_P^{(\ell)} = \tanh \left(\frac{2}{K+1} \sum_{k=0}^K \sum_{j=0}^K \hat{\theta}_j P_k(\kappa_j) P_k(\hat{L}) Z_P^{(\ell)} \Theta_{att} \right) \quad (17)$$

where $\tanh$ is an activation function, $\Theta_{att} \in \mathcal{R}^{F_P^{(\ell)} \times 1}$ denotes the learnable parameters of the SAGPool layer. $\mathcal{D}_P^{(\ell)} \in \mathcal{R}^{Q \times 1}$ and $Z_P^{(\ell)} \in \mathcal{R}^{Q \times F_P^{(\ell)}}$ represent the output and input features of the ℓ -th graph pooling layer, respectively. $F_P^{(\ell)}$ represents the dimension of the input features of the ℓ -th graph convolutional layer in the SAGPool layer.

Self-attention mask: To reduce the number of graph nodes in graph signals, the SAGPool adopts the top-rank method to retain the top $\lceil \eta Q \rceil$ nodes, where η is a predefined parameter that determines the number of nodes to keep. Specifically, the self-attention scores are sorted in descending order. Then, the top $\lceil \eta Q \rceil$ nodes are retained based on the sorted self-attention scores, and the indices of the top $\lceil \eta Q \rceil$ scores are returned. Finally, the scores of the retained nodes form the mask matrix. The process of selecting graph nodes can be expressed as

$$\begin{aligned} idx &= \text{top-rank} \left(\mathcal{D}_P^{(\ell)}, \lceil \eta Q \rceil \right) \\ \mathcal{D}_{mask}^{(\ell)} &= \mathcal{D}_{idx}^{(\ell)} \end{aligned} \quad (18)$$

where the *top-rank* function returns the indices of the top $\lceil \eta Q \rceil$ self-attention scores. The *idx* denotes an indexing operation. $\mathcal{D}_{mask}^{(\ell)} \in \mathcal{R}^{\lceil \eta Q \rceil \times 1}$ denotes the feature attention mask matrix, which consists of self-attention values extracted by the *idx*.

Self-attention pooling: The final graph pooling operation can be implemented by applying the mask matrix to the input graph signal through a masking operation. Specifically, node features are extracted row-wise (node-wise) to form the indexed feature matrix. Then, the extracted node features and attention mask matrix implement a dot product operation using a broadcast mechanism to obtain the pooled node features. Finally, the pooled adjacency matrix is obtained by extracting row-wise and col-wise the original adjacency matrix. The process of self-attention graph pooling can be denoted as

$$\begin{aligned} \hat{Z}_P^{(\ell)} &= Z_P^{(\ell)}_{idx,:} \\ Z_P^{(\ell+1)} &= \hat{Z}_P^{(\ell)} \odot \mathcal{D}_{mask}^{(\ell)} \\ A_P^{(\ell+1)} &= A_{idx,idx}^{(\ell)} \end{aligned} \quad (19)$$

where $Z_P^{(\ell)}_{idx,:}$ represents the row-wise indexed feature matrix, and $A_{idx,idx}^{(\ell)}$ denotes the row-wise and col-wise indexed adjacency matrix. $Z_P^{(\ell+1)} \in \mathcal{R}^{(\lceil \eta Q \rceil \times F_P^{(\ell+1)})}$ and

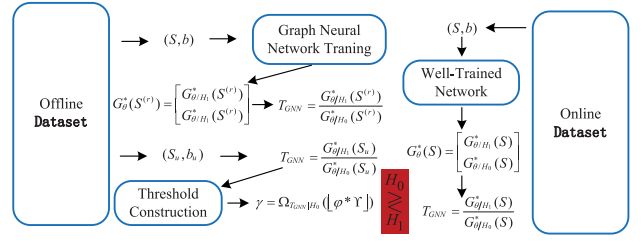


Fig. 3. The offline training scheme and online sensing scheme.

$A_P^{(\ell+1)} \in \mathcal{R}^{(\lceil \eta Q \rceil \times \lceil \eta Q \rceil)}$ represent the pooled node features and pooled adjacency matrix, respectively.

3) **Readout Layer:** Due to the stack of graph convolutions, the output features of deep graph neural networks may be over-smoothing and the nodes from different clusters may become indistinguishable. To mitigate over-smoothing, a readout layer has been proposed to aggregate node features into a fixed-size representation and can be denoted as

$$Z_R^{(\ell+1)} = \frac{1}{\hat{Q}} \sum_{i=1}^{\hat{Q}} Z_{R,i}^{(\ell)} \parallel \hat{m}_{i=1}^{\hat{Q}} Z_{R,i}^{(\ell)} \quad (20)$$

where $Z_R^{(\ell)} \in \mathcal{R}^{\hat{Q} \times F_R^{(\ell)}}$ and $Z_R^{(\ell+1)} \in \mathcal{R}^{1 \times 2F_R^{(\ell)}}$ represent the input and output features of the readout layer, where $\hat{Q}$ and $F_R^{(\ell)}$ represent the number of nodes after graph pooling and the feature dimension of the readout layer, respectively. $Z_{R,i}^{(\ell)} \in \mathcal{R}^{1 \times F_R^{(\ell)}}$ denotes the i -th vector of the node features. The symbol $\parallel$ represents the concatenation operation.

4) **Fully Connected Layer and Softmax Layer:** The work utilizes a multilayer perceptron network for the classification task, which is fully connected to the SAGPool layer. The multilayer perceptron network comprises two fully connected layers: FC1 and FC2. The single fully connected layer can be expressed as

$$Z_F^{(\ell+1)} = \sigma \left(Z_F^{(\ell)} W_F^{(\ell+1)} \right) \quad (21)$$

where $Z_F^{(\ell+1)} \in \mathcal{R}^{1 \times F_F^{(\ell+1)}}$, $Z_F^{(\ell)} \in \mathcal{R}^{1 \times F_F^{(\ell)}}$ and $W_F^{(\ell+1)} \in \mathcal{R}^{F_F^{(\ell)} \times F_F^{(\ell+1)}}$ represent the output features, input features, and weighted matrix of the ℓ -th fully connected layer, respectively.

Since spectrum sensing problem can be regarded as a binary classification task, the final output of the softmax layer should be a 2×1 score vector, denoted as

$$\text{softmax} \left(Z_i^{(\ell)} \right) = \frac{e^{Z_i^{(\ell)}}}{\sum_{j=1}^2 e^{Z_j^{(\ell)}}}, i = 1, 2 \quad (22)$$

C. Graph Neural Network-Based Spectrum Sensing Algorithm

In this section, the offline training and online detection modules of the SCM-GNN algorithm will be introduced, as shown in Fig. 3.

1) **Offline Training:** For the offline training module, the labeled sensing samples are gathered to construct the training dataset. The constructed dataset is represented as

$$\Xi = \left\{ \left(S^{(1)}, b^{(1)} \right), \left(S^{(2)}, b^{(2)} \right), \dots, \left(S^{(\Upsilon)}, b^{(\Upsilon)} \right) \right\} \quad (23)$$

where Ξ represents the collection of labeled covariance matrices. The $(S^{(r)}, b^{(r)})$ indicates the r -th labeled example in the training dataset. Therefore, $S^{(r)} \in \mathcal{R}^{M \cdot \zeta \times M \cdot \zeta}$ represents the covariance matrix with smooth factor. The variable $b^{(r)} \in \{0, 1\}$ denotes the label of received signals, where $b^{(r)} = 1$ and $b^{(r)} = 0$ represent the presence and absence of a PU signal respectively.

The labeled sensing sample is fed into the SCM-GNN algorithm, and the output score vector is calculated using equation (24). Therefore, the normalized softmax score vector is represented as

$$G_{\theta}(S^{(r)}) = \begin{bmatrix} G_{\theta/H_1}(S^{(r)}) \\ G_{\theta/H_0}(S^{(r)}) \end{bmatrix} \quad (24)$$

$$G_{\theta/H_1}(S^{(r)}) + G_{\theta/H_0}(S^{(r)}) = 1$$

where $G_{\theta}(S^{(r)})$ represents the expression of the SCM-GNN with model parameters θ . The $G_{\theta/H_i}(S^{(r)})$ indicates the class probability of hypothesis H_i .

An appropriate loss function plays a crucial role in model training and determines the effectiveness of the trained model. In this work, cross entropy is selected as the loss function. Therefore, for a minibatch dataset incorporating N_B labeled samples, the loss function is given by equation (25)

$$\mathcal{L} = -\frac{1}{N_B} \sum_{r=1}^{N_B} \left[b^{(r)} \log(G_{\theta}(S^{(r)})) + (1 - b^{(r)}) \log(1 - G_{\theta}(S^{(r)})) \right] \quad (25)$$

The goal of model training is to minimize the loss function and achieve the optimal expression $G_{\theta}(S^{(r)})$ for model deployment, that is,

$$\theta^* = \operatorname{argmin}_{\theta}(\mathcal{L}(\theta)) \quad (26)$$

The backpropagation algorithm iteratively updates the parameters of the graph neural network, resulting in a trained model. According to Neyman-Pearson's theorem, the optimal decision statistic can be denoted as

$$T_{GNN} = \frac{G_{\theta^*/H_1}^*(S^{(r)})}{G_{\theta^*/H_0}^*(S^{(r)})} \quad (27)$$

2) *Online Detection*: After calculating the decision statistic, determining an appropriate detection threshold γ is crucial for generating the receiver operating characteristic curve. Generally, the state of the primary user is inferred from the maximum value in the output probability vector of the graph neural network. However, the probability of false alarm (PFA) cannot be predetermined. Therefore, a Monte Carlo method is employed to determine a detection threshold γ that achieves the required PFA. The specific implementation details of the Monte Carlo method are presented in **Algorithm 1**. After obtaining the required detection threshold, the state of the primary user can be determined by

$$T_{GNN} \gtrless_{H_0}^{H_1} \gamma \quad (28)$$

Algorithm 1 Construction of Detection threshold

Input : Training dataset Ξ , Probability of false alarm φ , The well-trained neural network $G_{\theta^*/H_i}^*(\cdot)$.
Output : The detection threshold γ

1. Collect the samples under hypothesis H_0 to construct new dataset $(\Xi, T) = \left\{ \left(S_u^{(1)}, 0 \right), \dots, \left(S_u^{(\Upsilon)}, 0 \right) \right\}$
2. Feed the dataset into the trained network to obtain the decision statistic $T_{GNN}|_{H_0}$ under H_0
3. Sort decision statistics in descending order and construct the decision statistic dataset $(\Omega_{T_{GNN}}, 0)$
4. Get the detection threshold $\gamma = \Omega_{T_{GNN}|_{H_0}}(\lfloor \varphi * \Upsilon \rfloor)$, where $\lfloor \cdot \rfloor$ is round down

V. NUMERICAL SIMULATION RESULTS

In this section, we provide a detailed illustration of the detection performance of the proposed method through numerical simulation experiments. First, the simulation settings are described. Then, the experimental results are evaluated under various scenarios, including noise, channel fading, noise uncertainty, real-world condition, imperfect reporting channel, and robustness. Finally, the computational complexity of all algorithms is calculated for a fair comparison.

A. Simulation Setting

In this work, we evaluate the detection performance by comparing the SCM-GNN method with ED [6], MED [7], ME-AM [12], CM-CNN [20], DS2MA [28] and GCN [29] in terms of Receiver Operating Characteristics (ROC) curves and PD versus SNR (PD-SNR) curves.

1) *Sensing Data Generation*: In the simulation configuration, the SU is equipped with 8 receiving antennas ($M = 8$), and the correlation coefficient ρ between adjacent antennas is set to 0.5. In the generation of PU signals, the work employs quadrature phase shift keying (QPSK) as the modulation scheme. The pulse shaping filter uses a root raised cosine filter with a roll-off factor of 0.5, a symbol span of 4, and 8 samples per symbol. The symbol rate is set to 1 MHz, and the measured bandwidth of PU signals is 1.5 MHz. For all simulation experiments, the number of modulated symbols is set to 100 in each sensing period. The bit information stream is randomly generated and transmitted after modulation and shaping. For noise generation, the additive noise on each receiving antenna is assumed to obey a Gaussian distribution with zero mean and variance σ_n^2 . Additionally, we assume that the noise power remains consistent on each antenna. Therefore, the signal-to-noise ratio (SNR) [23] is given by equation (29).

$$SNR = \frac{\sigma_u^2}{\sigma_n^2} = \frac{E(\|s(n)\|^2)}{E(\|w(n)\|^2)} \quad (29)$$

The signal power can be expressed as $\sigma_u^2 = SNR \cdot \sigma_n^2$, where the noise power σ_n^2 is set to 1 W (0 dB). The noise power σ_n^2 is set to -174 dBm/Hz under the large-scale fading scenario.

TABLE I
THE HYPERPARAMETERS OF SCM-GNN

<i>Input: $X(\text{Dimension: } 2 \times 64 \times 64)$</i>				
Layer	Input Shape	Output Shape	Kernel Size	Trainable Params
Conv+BN+ReLU	$2 \times 64 \times 64$	$32 \times 7 \times 7$	(16×16)	16480
Graph signal Construction	$32 \times 7 \times 7$	49×32	-	-
ChebNetII+ReLU	49×32	49×32	(32×32)	1056
ChebNetII+ReLU	49×32	49×32	(32×32)	1056
ChebNetII+ReLU	49×32	49×32	(32×32)	1056
Concat	-	49×96	-	-
SAGE Pooling	49×96	49×96	-	97
Readout Layer	49×96	1×192	-	-
FC Layer1	192	64	(192×64)	12352
FC Layer2	64	2	(64×2)	130
Total Parameters				32,227

The SNR of the datasets ranges from -30 dB to -16 dB, with an interval of 2 dB. The number of labeled samples is set to 2,000 at each SNR.

2) *Simulation Environment and Hyperparameters:* The training and testing of the proposed algorithm are performed in the PyTorch framework, executing on an Nvidia 2080Ti GPU. The Adam optimizer with gradient clipping is used to optimize model parameters, where the clipping factor is set to 1. During the training of the SCM-GNN algorithm, the mini-batch size is set to 200, and the initial learning rate is set to 0.001. During the training of benchmark algorithms, the cross entropy and Adam optimizer are used as the loss function and optimizer, respectively. The mini-batch is set to 200. The initial learning rates for GCN, DS2MA, and CM-CNN are set to 0.001, 0.0001, and 0.0001, respectively.

In the SCM-GNN method, we assume that $M = 8$, $\zeta = 8$, $N_{patch} = 2M = 16$, $S_{patch} = M = 8$, $H = W = ((M \cdot \zeta - N_{patch}) / S_{patch} + 1) = 7$, $Q = H \cdot W = 49$, $F = F_C^{(\ell)} = 32$, $F_P^{(l)} = 96$. Therefore, the covariance matrix with smooth factor $R_y \in \mathcal{R}^{2 \times M \cdot \zeta \times M \cdot \zeta}$ is of dimensions $2 \times 64 \times 64$. The graph node features $X \in \mathcal{R}^{Q \times F}$ and adjacency matrix $A \in \mathcal{R}^{Q \times Q}$ are of dimensions 49×32 and 49×49 respectively. In the GCN method, the test statistic consists of the graph signal which include the node features and an adjacency matrix. The node features are obtained by calculating RSS and the elements in the adjacency matrix are calculated by radial basis function. Therefore, the graph node features and adjacency matrix are of dimensions $M \times 1$ and $M \times M$, respectively. In the CM-CNN method, the test statistic is derived from the auto-covariance matrix and of dimensions $2 \times M \times M$. The auto-correlation and cross-correlation features are used as the test statistic in DS2MA method. The test statistic is of dimensions $2 \times (M + M_{C_2}) \times L_\tau$, where L_τ is set as 25.

In the GCN architecture, except for the first graph convolution layer where the dimensions of the input and output features are 1 and 32, respectively, each graph convolution layer adopts an MLP function that doubles the dimension of input features. Global sum pooling is employed to aggregate information from all nodes, reducing the number of graph nodes to 1. A kernel size of 5×5 and a stride length of 1 are

TABLE II
THE HYPERPARAMETERS OF GCN

<i>Input: $X(\text{Dimension: } 8 \times 1)$</i>				
Layer	Input Shape	Output Shape	Kernel Size	Trainable Params
GCNConv1+ReLU	8×1	8×32	(1×32)	64
GCNConv2+ReLU	8×32	8×64	(32×64)	2112
GCNConv3+ReLU	8×64	8×128	(64×128)	8320
Global Sum Pooling	8×128	1×128	-	97
FC Layer1+ReLU	128	64	(192×64)	8256
FC Layer2+Softmax	64	2	(64×2)	130
Total Parameters				18,882

TABLE III
THE HYPERPARAMETERS OF CM-CNN

<i>Input: $X(\text{Dimension: } 2 \times 8 \times 8)$</i>				
Layer	Input Shape	Output Shape	Kernel Size	Trainable Params
Conv1+ReLU	$2 \times 8 \times 8$	$20 \times 8 \times 8$	(5×5)	1020
Max Pooling1	$20 \times 8 \times 8$	$20 \times 4 \times 4$	-	-
Conv2+ReLU	$20 \times 4 \times 4$	$50 \times 2 \times 2$	(5×5)	25050
Max Pooling2	$50 \times 2 \times 2$	$50 \times 1 \times 1$	-	-
FC Layer1	50	500	(50×500)	25500
FC Layer2	500	2	(500×2)	1002
Total Parameters				52,572

TABLE IV
THE HYPERPARAMETERS OF DS2MA

<i>Input: $X(\text{Dimension: } 2 \times 36 \times 25)$</i>				
Layer	Input Shape	Output Shape	Kernel Size	Trainable Params
Conv1+ReLU	$2 \times 36 \times 25$	$128 \times 36 \times 25$	(3×3)	2432
Conv2+ReLU	$128 \times 36 \times 25$	$128 \times 36 \times 25$	(3×3)	147584
Max-Pooling	$128 \times 36 \times 25$	$128 \times 18 \times 12$	(2×2)	-
Conv3+ReLU	$128 \times 18 \times 12$	$128 \times 18 \times 12$	(3×3)	147584
Conv4+ReLU	$128 \times 18 \times 12$	$128 \times 18 \times 12$	(3×3)	147584
Flatten	$128 \times 18 \times 12$	1×27648	-	-
FC Layer1	27648	128	(27648×128)	3529072
FC Layer2	128	2	(128×2)	258
Total Parameters				3,984,514

adopted for CM-CNN in each convolution layer. The width and height of the output features of the two convolution layers are maintained constant by zero padding. In DS2MA, each convolutional layer applies a kernel size of 3×3 , a stride length of 1, and zero padding to maintain constant dimensions for the input and output features. Each max pooling layer in CM-CNN and DS2MA is used to halve the width and height of the input features. Table I represents the architectural details and model parameters of SCM-GNN. Tables II, III, and IV illustrate the architectural details and model hyperparameters of the GCN, CM-CNN, and DS2MA methods, respectively.

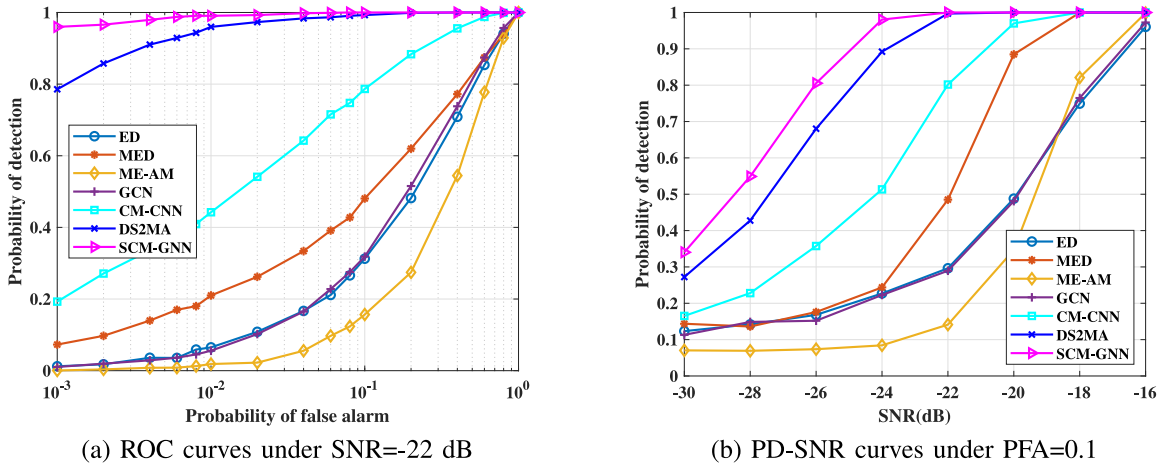


Fig. 4. Detection performance of different detection algorithms under AWGN.

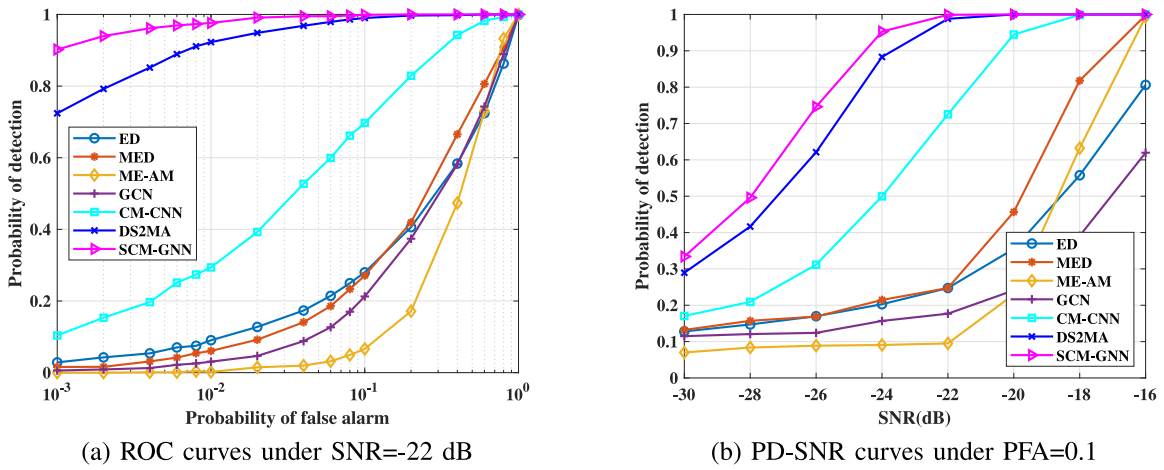


Fig. 5. Detection performance of different detection algorithms under CGGN.

B. Simulation Results Under Noise

In this section, the detection performance of the SCM-GNN algorithm is evaluated under the noise scenario.

1) *Additive White Gaussian Noise (AWGN)*: Firstly, we evaluate the detection performance under AWGN. The AWGN follows a circularly symmetric complex Gaussian distribution with zero mean and covariance σ^2 . Fig. 4(a) shows the ROC curves of these algorithms at the detection SNR of -22 dB. It is evident that SCM-GNN exhibits a higher detection probability compared to other benchmark algorithms. In contrast to conventional model-based algorithms, deep learning-based algorithms, except for the GCN method, demonstrate superior detection performance. The GCN method uses RSS as input statistics, which cannot provide additional effective information to improve detection performance compared to other test statistics. To better evaluate the detection performance of SCM-GNN, the area under the ROC curves (AUC) is calculated. The AUC values for SCM-GNN, DS2MA, CM-CNN, GCN, MED, ME-AM, and ED are 0.9986, 0.9962, 0.9232, 0.7257, 0.7682, 0.6068, and 0.7069, respectively. It can be observed that SCM-GNN exhibits the highest sensing accuracy.

Fig. 4(b) denotes the PD-SNR curves of the detection algorithms under AWGN. The detection SNR ranges from -30 dB to -16 dB. The figure illustrates that the SCM-GNN method achieves optimal detection performance, demonstrating significant enhancements in the achievable PD values compared to the ED, MED, ME-AM, GCN, and CM-CNN methods. In contrast to other benchmarks, DS2MA closely matches the detection performance of the SCM-GNN method in terms of ROC curves and PD-SNR curves. It can be interpreted that both DS2MA and SCM-GNN can effectively capture the correlation among adjacent antennas, which influences the probability of successful PU detection.

2) *Complex Generalized Gaussian Noise (CGGN)*: The CGGN follows a complex generalized Gaussian distribution (CGGD), as described in Section II-B1. The implementation details of CGGN are shown in Algorithm 2. The shape parameter s , scale parameter γ_g , and the correlation coefficient ρ of the distribution are set to 0.4, 1, and 0, respectively. Furthermore, the generalized SNR under CGGN can be defined as $SNR = P_{pu}/(2\gamma_g)$.

Fig. 5(a) and Fig. 5(b) show the ROC curves and PD-SNR curves, respectively. It can be observed that the PD values of the SCM-GNN algorithm are superior

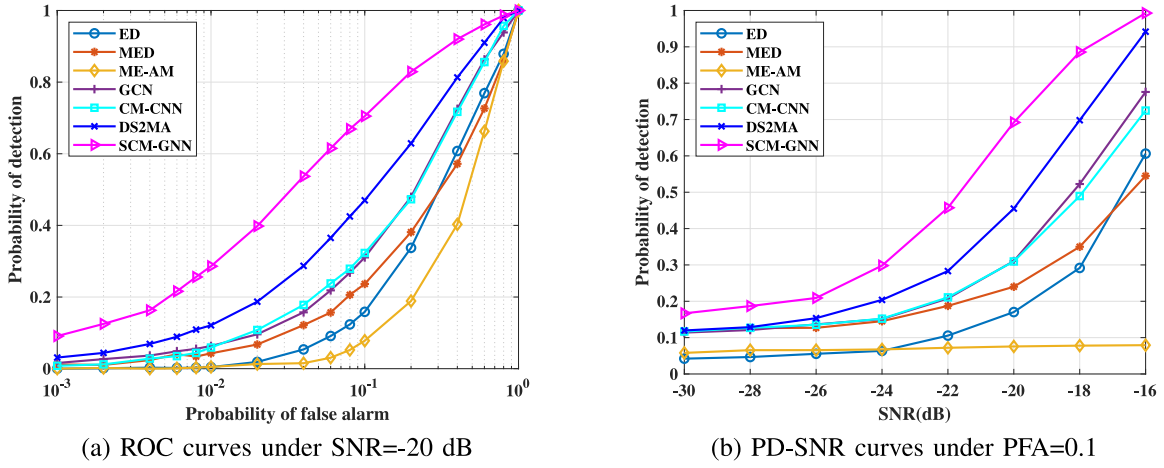


Fig. 6. Detection performance of different detection algorithms under FWGN fading.

Algorithm 2 Generating CGGN Noise

Input : Shape parameter s ; Scale parameters γ_g ;
Correlation coefficient ρ

Output : Complex generalized Gaussian noise v

1. Generate $z_0 = \Gamma(1/s, 1)^{1/2s} e^{(j2\pi u(0,1))}$;
2. Normalize $z_0 = z_0 / \sqrt{\frac{\Gamma(2/c)}{\Gamma(1/c)}}$;
3. Form augmented vector $w = [z_0, z_0^*]^T$;
4. Calculate transform $T = \begin{pmatrix} 2\gamma_g & 2\rho j \\ -2\rho j & 2\gamma_g \end{pmatrix}^{1/2}$;
5. Generate noise $v = Tw$;

to that of benchmark algorithms, demonstrating its effectiveness under CGGN. Given a specific SNR constraint of -24 dB, the SCM-GNN algorithm achieves a PD value of 0.9530, outperforming DS2MA(0.8831), CM-CNN(0.4995), GCN(0.1568), MED(0.2146), ED(0.2027), and ME-AM(0.0907). Furthermore, compared to traditional detection algorithms, the SCM-GNN, DS2MA, and CM-CNN algorithms can effectively mitigate the impact of CGGN. In contrast to the detection performance under AWGN, all methods experience varying degrees of degradation in detection performance under CGGN. This degradation is related to the heavy-tailed property of CGGN. Due to its heavy-tailed property, it is inevitable for CGGN to introduce impulsive noise that weakens the discriminating features of input statistics and increases the probability of false detection.

C. Simulation Results Under Fading Channels

During signal transmission, channel fading inevitably results in signal distortion. To further evaluate detection performance under fading channels, two commonly used channels are considered: Rayleigh channel and Nakagami- m channel.

1) *Rayleigh Fading Channel:* The work utilizes filtered white Gaussian noise (FWGN) [35] to simulate the Rayleigh fading channel. For the FWGN channel, the maximum Doppler shift and Doppler spectrum are set to 100 Hz and the classical Doppler spectrum, respectively. Fig. 6(a) illustrates the ROC

curves of different algorithms under the FWGN channel. It is evident that the detection performance of all algorithms deteriorates significantly compared to their detection performance under AWGN. However, the SCM-GNN algorithm still maintains the highest detection accuracy compared to other benchmark algorithms. For example, at $PFA = 0.1$, the SCM-GNN algorithm achieves a PD of 70.5%, outperforming DS2MA(47%), CM-CNN(31.05%), GCN(32.3%), MED(23.65%), ED(15.86%), and ME-AM(7.80%), which demonstrates its superiority under the FWGN channel.

The PD-SNR curves of the SCM-GNN algorithm along with other benchmark algorithms are shown in Fig. 6(b) under the FWGN channel. It can be concluded from the figure that the SCM-GNN algorithm outperforms both deep learning-based and model-based algorithms in terms of detection performance across different SNRs. Compared to DS2MA and CM-CNN, the SCM-GNN can bring up to 2 dB and 4 dB of detection gain, respectively. In contrast to ED and MED, the SCM-GNN provides at least 4 dB gain.

2) *Complex Nakagami- m Fading Channel:* The complex Nakagami- m fading is used to simulate the effects of a fast fading channel. The fading parameter m , phase parameter p , and power parameter Ω are set to 0.8, 0.2, and 1, respectively. The ROC curves and PD-SNR curves are illustrated in Fig. 7(a) and Fig. 7(b) under the Nakagami fading channel, respectively. Although these curves reveal the performance degradation for all algorithms under Nakagami fading, the SCM-GNN method still maintains the highest detection probability. Given a specific SNR constraint of -22 dB, the SCM-GNN algorithm achieves a PD value of 22.18%, outperforming DS2MA(16.80%), CM-CNN(18.12%), GCN(18.91%), MED(13.72%), ED(10.17%), and ME-AM(7.10%). In terms of AUC calculation, the SCM-GNN method exhibits superior performance with an AUC value of 0.7053, surpassing CM-CNN (0.6516) and DS2MA (0.6755) by 0.0298 and 0.0537, respectively. Furthermore, the AUC value of the GCN method is 0.6975, demonstrating comparable detection performance with the SCM-GNN method. It can be seen that GNN-based algorithms effectively mitigate the impact of Nakagami fading.

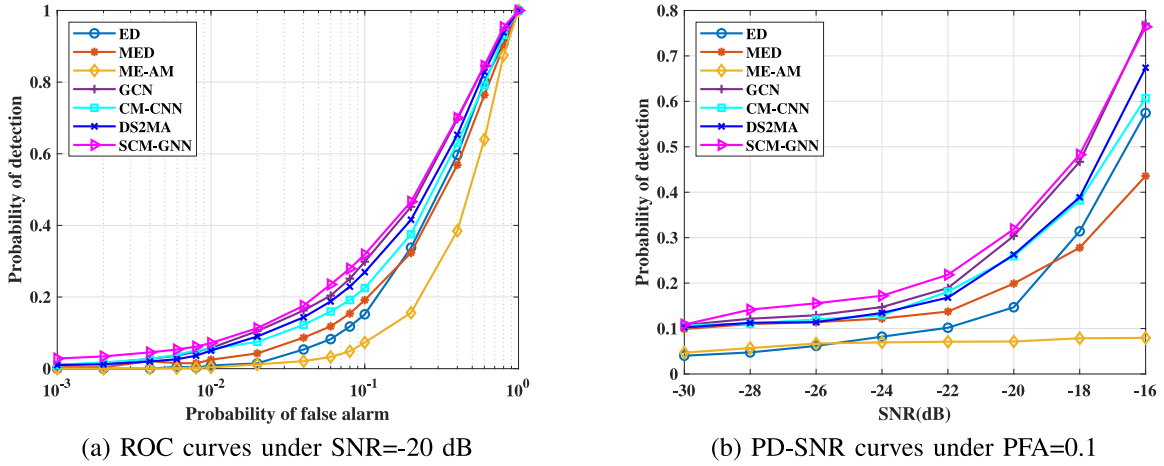


Fig. 7. Detection performance of different detection algorithms under Nakagami-m fading.

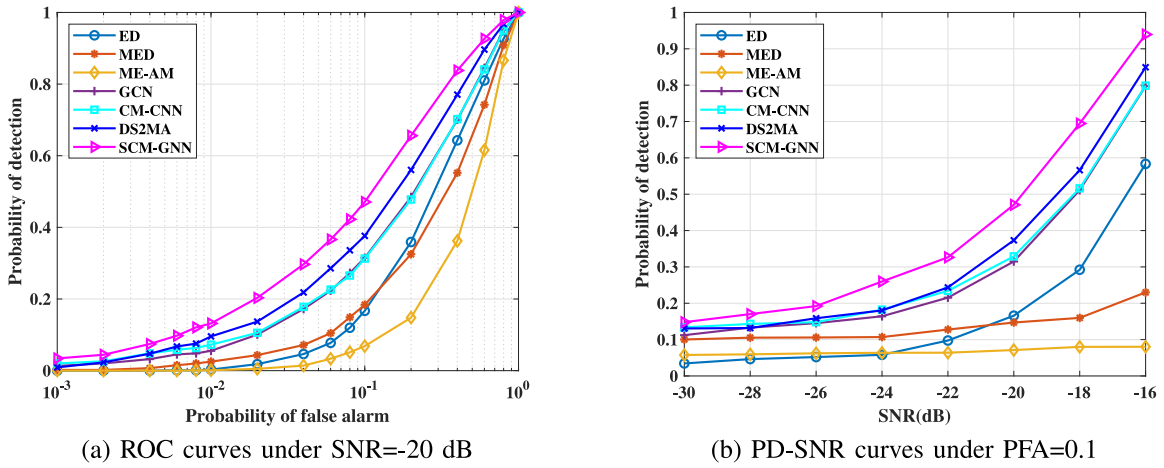


Fig. 8. Detection performance of different detection algorithms under large-scale fading.

D. Simulation Results Under Large-Scale Fading

In this section, we evaluate the impact of large-scale fading on the detection performance. We assume that $P = 23$ dBm, $\beta = 10^{3.453}$, $\alpha = 3.8$, $\sigma = 7.9$ dB, and $d = 200$ m. Fig. 8(a) shows the ROC curves of the SCM-GNN method and other benchmark schemes at a detection SNR of -20 dB. Despite the performance degradation of all methods due to large-scale fading, the SCM-GNN algorithm still demonstrates superior ROC curves. In terms of AUC values, the AUC value of the SCM-GNN algorithm (0.8016) surpasses those of DS2MA (0.7534), CM-CNN (0.7049), GCN (0.7063), ME-AM (0.4972), MED (0.6078), and ED (0.6454), which demonstrates its superiority under large-scale fading.

The PD-SNR curves of the SCM-GNN algorithm, along with those of benchmark algorithms, are depicted in Fig. 8(b). Consistent with the conclusions drawn from previous experiments, the SCM-GNN algorithm exhibits superior detection performance compared to the benchmark algorithms. Given a specific SNR constraint of -20 dB, the SCM-GNN algorithm achieves a PD value of 48.58%, outperforming DS2MA (37.32%), CM-CNN (32.89%), GCN (31.48%), MED (14.70%), ED (16.57%), and ME-AM (7.12%). Furthermore, Fig. 8(b) indicates that the detection performance of

model-based methods (MED, ED, ME-AM) deteriorates significantly compared to that of deep learning-driven methods. Compared to other deep learning-based detection methods, the SCM-GNN method provides at least 1 dB detection gain.

E. Simulation Results Under Noise Uncertainty

In practice, the noise variance dynamically varies over different sensing periods, resulting in noise uncertainty. As described in Section II-B3, the noise uncertainty coefficient is set to 0.5. Fig. 9 shows the PD-SNR curves under noise uncertainty. The PD-SNR curves reveal the considerable drop in detection performance for MED and ED methods as compared to their performance under AWGN. The ME-AM algorithm, which is categorized as a totally blind detection algorithm, exhibits consistent detection performance regardless of noise uncertainty. In addition, deep learning-based algorithms effectively mitigate the effects of noise uncertainty by learning discriminative features from a diverse set of examples, enhancing the capability to detect the state of PU. The detection performance of GCN seriously degrades in the presence of noise uncertainty, primarily due to its inability to effectively distinguish between the RSS of noise signals and the RSS of PU signals.

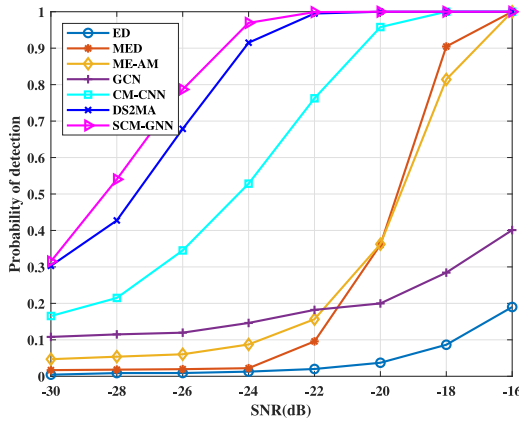


Fig. 9. PD-SNR curves of different detection algorithms under noise uncertainty at PFA = 0.1.

F. Simulation Results of Algorithm Stability

1) *Scalability Evaluation*: The scalability of the SCM-GNN algorithm is evaluated through a series of ablation experiments involving varying numbers of modulated symbols and receiving antennas. The scalability evaluation is conducted in multiple channel environments, including AWGN, CGGN, Rayleigh fading, and large-scale fading.

Considering the number of modulated symbols as 50, 100, and 150, the lengths N of received signals are 425, 825, and 1225, respectively. The SCM-GNN algorithm is trained using a single training dataset for each channel scenario, which comprises three types of signals with varying lengths. Fig. 10(a) illustrates the ROC curves for different numbers of modulated symbols under multiple channel scenarios. It is evident that the detection performance of the SCM-GNN algorithm follows a trend of improvement with an increase in the number of modulated symbols under each channel scenario. Compared to its performance under imperfect channels, the detection performance of SCM-GNN exhibits a significant improvement under perfect channels. It can be inferred that the proposed algorithm demonstrates scalability under different numbers of modulated symbols.

The work also evaluates the scalability of the SCM-GNN algorithm with different numbers of receiving antennas across multiple channel scenarios. Considering the number M of receiving antennas as 4, 8, and 16, the dimensions of the test statistic are $(2 \times 32 \times 32)$, $(2 \times 64 \times 64)$, and $(2 \times 128 \times 128)$, respectively. Fig. 10(b) depicts the PD-SNR curves for different numbers of receiving antennas across multiple channel scenarios. The observed trend indicates an improvement in the detection performance of the proposed method as the number of receiving antennas increases. Furthermore, the detection performance of SCM-GNN exhibits a significant degradation under channel fading (Rayleigh fading and Large-scale fading) compared to its performance under noise conditions (AWGN and CGGN). It can be seen that the SCM-GNN algorithm is scalable for different multi-antenna systems.

2) *Generalization Evaluation*: The generalizability of the SCM-GNN algorithm is evaluated through ablation experiments, in which the trained SCM-GNN algorithm is tested

across various channel environments. During the generalization evaluation, the SCM-GNN algorithm is pre-trained and does not require fine-tuning under different testing scenarios.

Fig. 11(a) illustrates the ROC curves of the generalization evaluation under various tested scenarios. The PD-SNR curves are described in Fig. 11(b) under various tested scenarios. Fig. 11(a) and Fig. 11(b) demonstrate that the well-trained SCM-GNN algorithm can directly perform spectrum sensing tasks without retraining the neural network. It can be inferred that the SCM-GNN method exhibits a certain level of generalizability under diverse tested scenarios. However, the detection performance of the algorithm experiences performance degradation, particularly in tested scenarios including Rayleigh fading, large-scale fading, and Nakagami fading, with performance degradation reaching up to 10 %.

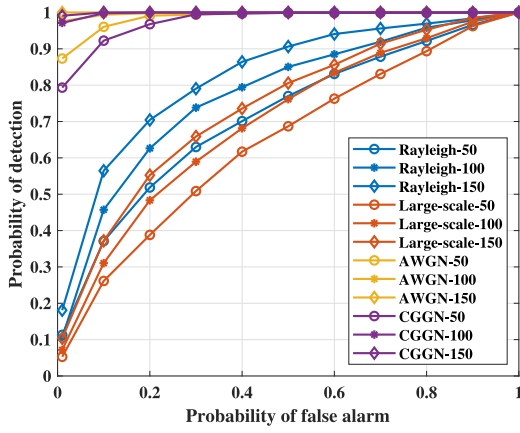
G. Simulation Results Under Real-World Scenario

The effectiveness of the proposed algorithm is evaluated in real-world scenarios using sensing samples collected with the USRP B200 mini device. Due to limitations in the number of RF channels on the USRP device, the USRP B200 mini can only utilize a single pair of RF transmit/receive channels for sending and receiving RF data. Therefore, the proposed SCM-GNN algorithm transforms from a multi-antenna spectrum sensing algorithm to a single-antenna spectrum sensing algorithm. In terms of the construction of a real-world dataset, with the help of a spectrum analyzer, channels with high SNR are identified and clean PU signals are captured by USRP devices. Background noise in the real-world scenario is also collected as samples under H_0 . To obtain sensing samples with a given SNR under H_1 , the required power of PU signals is calculated using the relation $\sigma_u^2 = SNR \cdot \sigma_n^2$, and the PU signals at this power level are added to the background noise.

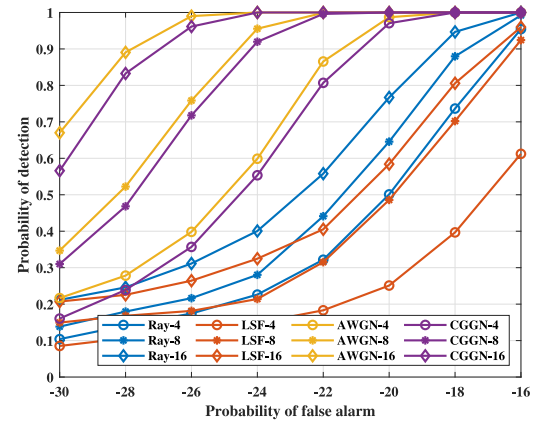
The benchmark algorithms based on deep learning, such as DS2MA, CM-CNN, and GCN, are designed for multi-antenna spectrum sensing and are not applicable for single-antenna spectrum sensing. Therefore, the CLDNN algorithm is introduced as a contrast algorithm based on deep learning. The ROC curves and PD-SNR curves are illustrated in Fig. 12(a) and Fig. 12(b) under the real-world scenario, respectively. Fig. 12(a) clearly shows that the SCM-GNN method exhibits a higher detection probability compared to these contrast algorithms. Furthermore, there are significant gaps in achievable PD values between the SCM-GNN algorithm and other benchmark methods. Fig. 12(b) clearly demonstrates that the SCM-GNN method maintains optimal detection performance, and brings at least 4 dB detection gain. In terms of AUC calculation, the AUC values for SCM-GNN, CLDNN, MED, ME-AM, and ED are calculated as 0.9356, 0.8497, 0.6859, 0.6568, and 0.7402, respectively. These AUC values demonstrate that the SCM-GNN algorithm demonstrates the superior performance under the real-world scenario.

H. Simulation Results Under Imperfect Reporting Channel

We have evaluated the effectiveness of the proposed SCM-GNN method under imperfect sensing channel and imperfect reporting channel. The received sensing signals from the

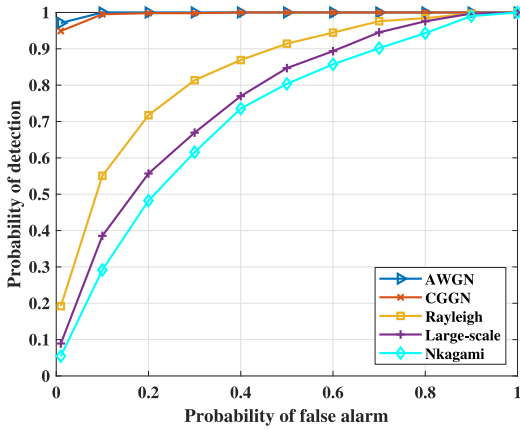


(a) ROC curves under SNR=-22 dB

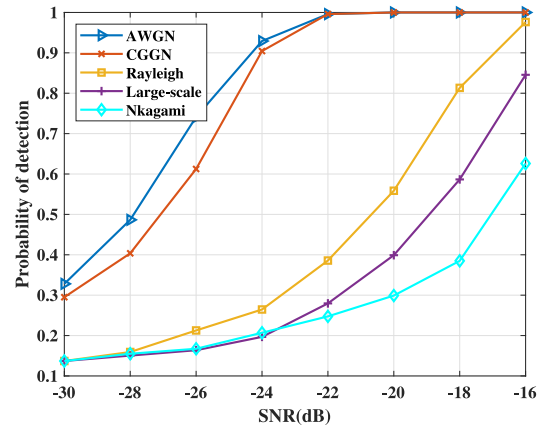


(b) PD-SNR curves under PFA=0.1

Fig. 10. Scalability of the proposed algorithm under different numbers of samples and antennas.

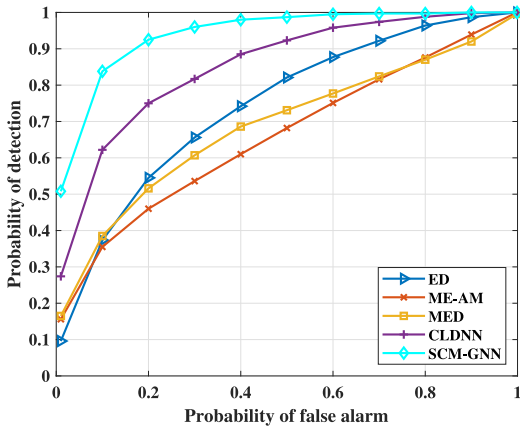


(a) ROC curves under SNR=-20 dB

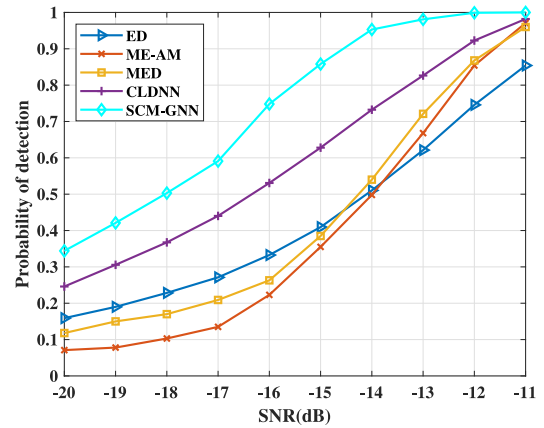


(b) PD-SNR curves under PFA=0.1

Fig. 11. Generalization evaluation of the proposed algorithm.



(a) ROC curves under SNR=-15 dB



(b) PD-SNR curves under PFA=0.1

Fig. 12. Detection performance of different detection algorithms under Real-World scenario.

primary user are inevitably disturbed by large-scale fading, and the sensing outcomes of the secondary user reported to the fusion center can experience small-scale fading. The fusion center is equipped with M receiving antennas to capture the sensing outcomes. Therefore, the transmission of sensing outcomes between the secondary user and the fusion center can be formulated as a Multi-Input-Multi-Output (MIMO)

scenario, and the received signal at the fusion center from the secondary user can be rewritten as

$$y_m^c(t) = \begin{cases} \sum_{i=1}^M h_{m,i}^c(t)(\kappa(t)h_i^s(t)s(t) + w_i^s(t)) + w_m^c(t) & H_1 \\ \sum_{i=1}^M h_{m,i}^c(t)w_i^s(t) + w_m^c(t) & H_0 \end{cases} \quad (30)$$

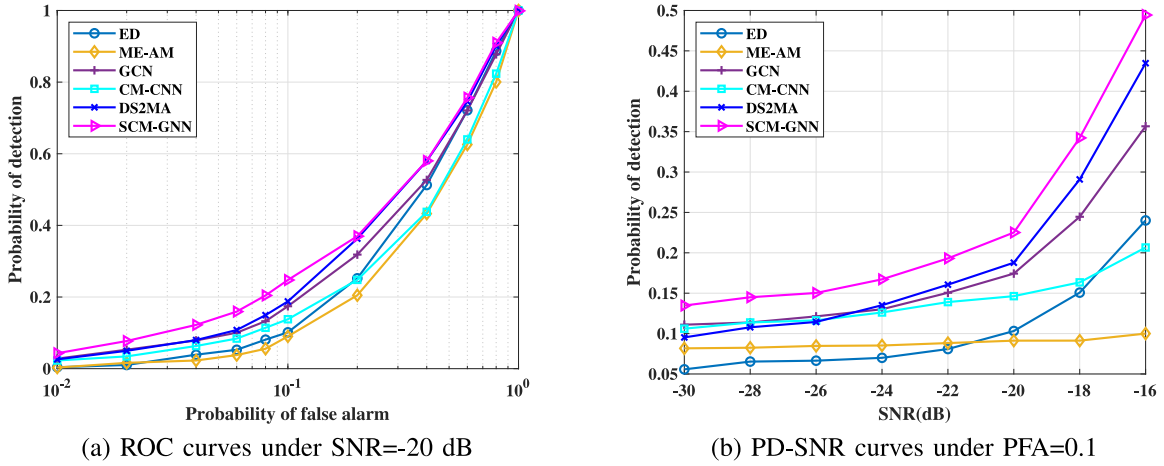


Fig. 13. Detection performance of different detection algorithms under Imperfect Reporting Channel.

where $h_{m,i}^c(t)$, and $h_i^s(t)$ represent the imperfect reporting channel gain between the i -th antenna of the secondary user and the m -th antenna of the fusion center, and the sensing channel gain at the i -th antenna of the secondary user, respectively. $w_m^c(t)$ and $w_i^s(t)$ denote the additive Gaussian white noise at the m -th antenna of the fusion center and the i -th antenna of the secondary user, respectively. $w_m^c(t)$ and $w_i^s(t)$ follow a circularly symmetric Gaussian distribution with zero mean and variance σ_n^2 . The work employs the FWGN model to simulate the imperfect reporting channel. The imperfect sensing channel is characterized by the large-scale fading described in section II-B2, with parameters consistent with those presented in Section V-C.

The ROC curves of the SCM-GNN method and other benchmark methods are depicted in Fig. 13(a) at a detection SNR of -20 dB. It can be observed from Fig. 13(a) that the detection performance of all methods deteriorates significantly compared to their performance under the perfect reporting channel. However, the SCM-GNN method still outperforms other benchmark methods, demonstrating superior ROC curves. Given a specific PFA value constraint of 0.1, the available PD value (0.2474) of the SCM-GNN algorithm surpasses those of DS2MA (0.1877), CM-CNN (0.1379), GCN (0.1742), ED (0.1014), and ME-AM (0.0901). The PD-SNR curves of the SCM-GNN algorithm, along with those of benchmark methods, are depicted in Fig. 13(b) under the imperfect reporting channel. The SCM-GNN algorithm still demonstrates superior detection performance compared to other benchmark methods. In terms of AUC values, the AUC value for SCM-GNN is 0.6306, outperforming DS2MA(0.6188), CM-CNN(0.5316), GCN(0.5905), ME-AM(0.5105), and ED(0.5718).

I. Complexity Analysis

Although the proposed algorithm achieves great detection performance, its computational complexity should be analyzed for a fair comparison. The work focuses on the computational complexity of model inference.

For model-based algorithms, the computational complexity of ED comes from the computation of the energy of received signals, which is expressed as $C_{ED} \sim O(MN)$.

The computational complexity of MED is divided into two parts: the computation of auto-correlations $O(M^2N)$ and the computation of maximum eigenvalue $O(M^3)$. Therefore, the computational complexity of MED is given by $C_{MED} \sim O(M^2N + M^3)$. The computation of the covariance matrix $O(M^2N)$, the eigenvalue decomposition $O(M^3)$, and the computation of geometric mean $O(M)$ constitute the computational complexity of ME-AM. Thus, the computational complexity is given by $C_{ME-AM} \sim O(M^2N + M^3 + M)$.

The computational complexity of CM-CNN and DS2MA is consistent, which consists of the computation of convolutional layers and fully connected layers. Therefore, we utilize C_{CNN} to represent the computational complexity of both CM-CNN and DS2MA. The computational complexity of CNN-based algorithms can be represented as

$$\begin{aligned} C_{CNN} &\sim C_{Conv} + C_{FC} \\ C_{Conv} &\sim O\left(\sum_{l=1}^{L_C} m_c^{(l-1)} \cdot (m_k^{(l)})^2 \cdot m_c^{(l)} \cdot (m_o^{(l)})^2\right) \\ C_{FC} &\sim O\left(\sum_{l=1}^{L_F} n_f^{(l-1)} \cdot n_f^{(l)}\right) \end{aligned} \quad (31)$$

where C_{Conv} and C_{FC} represent the computational complexity of convolutional layers and fully connected layers, respectively. L_C denotes the number of convolution layers. $m_c^{(l)}$ and $m_c^{(l-1)}$ denote the number of output and input channels in the l -th convolution layer, respectively. $m_k^{(l)}$ represents the convolution kernel size, and $m_o^{(l)}$ denotes the spatial size of the output feature map. $n_f^{(l)}$ and $n_f^{(l-1)}$ are the number of neurons in the l -th and $(l-1)$ -th fully connected layers, respectively. Although the computational complexity of CM-CNN and DS2MA is represented by C_{CNN} , their hyperparameters are inconsistent, leading to different results in computational complexity.

The computational complexity of GCN consists of the cost of graph convolutional layers and fully connected layers. The computational complexity C_{GCN} of GCN is defined as

TABLE V
THE COMPUTATIONAL COMPLEXITY OF DIFFERENT DETECTION ALGORITHMS

Algorithm	Online Detection	Offline Training	FLOPs(G)	Params(M)
ED	$O(MN)$	-	6.60×10^{-6}	-
MED	$O(M^2N + M^3)$	-	5.33×10^{-5}	-
ME-AM	$O(M^2N + M^3 + M)$	-	5.33×10^{-5}	-
CM-CNN	$O(\sum_{l=1}^{L_C} n_f^{(l-1)} \cdot (n_k^{(l)})^2 \cdot n_f^{(l)} \cdot (n_o^{(l)})^2 + \sum_{l=1}^{L_F} n_m^{(l-1)} \cdot n_m^{(l)})$	$O(\sum_{l=1}^{L_C} N_t \cdot N_e \cdot n_f^{(l-1)} \cdot (n_k^{(l)})^2 \cdot n_f^{(l)} \cdot (n_o^{(l)})^2 + \sum_{l=1}^{L_F} N_t \cdot N_e \cdot n_m^{(l-1)} \cdot n_m^{(l)})$	1.90×10^{-4}	5.25×10^{-2}
DS2MA	$O(\sum_{l=1}^{L_C} n_f^{(l-1)} \cdot (n_k^{(l)})^2 \cdot n_f^{(l)} \cdot (n_o^{(l)})^2 + \sum_{l=1}^{L_F} n_m^{(l-1)} \cdot n_m^{(l)})$	$O(\sum_{l=1}^{L_C} N_t \cdot N_e \cdot n_f^{(l-1)} \cdot (n_k^{(l)})^2 \cdot n_f^{(l)} \cdot (n_o^{(l)})^2 + \sum_{l=1}^{L_F} N_t \cdot N_e \cdot n_m^{(l-1)} \cdot n_m^{(l)})$	2.02×10^{-1}	3.98
GCN	$O(\sum_{l=1}^{L_G} E^{(l)} F_G^{(l)} + Q^{(l)} F_G^{(l)} F_G^{(l-1)} + \sum_{l=1}^{L_F} F_F^{(l-1)} F_F^{(l)})$	$O(\sum_{l=1}^{L_G} N_t \cdot N_e \cdot E^{(l)} F_G^{(l)} + Q^{(l)} F_G^{(l)} F_G^{(l-1)} + \sum_{l=1}^{L_F} N_t \cdot N_e \cdot F_F^{(l-1)} F_F^{(l)})$	9.85×10^{-5}	1.88×10^{-2}
SCM-GNN	$O(\sum_{l=1}^{L_C} n_f^{(l-1)} \cdot (n_k^{(l)})^2 \cdot n_f^{(l)} \cdot (n_o^{(l)})^2 + \sum_{l=1}^{L_G+L_P} (K^2 E^{(l)} F_G^{(l)} + Q^{(l)} (F_G^{(l)})^2) + \sum_{l=1}^{L_F} F_F^{(l-1)} F_F^{(l)})$	$O(\sum_{l=1}^{L_C} N_t \cdot N_e \cdot n_f^{(l-1)} \cdot (n_k^{(l)})^2 \cdot n_f^{(l)} \cdot (n_o^{(l)})^2 + \sum_{l=1}^{L_G+L_P} N_t \cdot N_e \cdot (K^2 E^{(l)} F_G^{(l)} + Q^{(l)} (F_G^{(l)})^2) + \sum_{l=1}^{L_F} N_t \cdot N_e \cdot F_F^{(l-1)} F_F^{(l)})$	8.95×10^{-4}	3.22×10^{-2}

$$\begin{aligned}
C_{GCN} &\sim C_{GC} + C_{FC} \\
C_{GC} &\sim O\left(\sum_{l=1}^{L_G} |E^{(l)}| F_G^{(l)} + Q^{(l)} F_G^{(l)} F_G^{(l-1)}\right) \\
C_{FC} &\sim O\left(\sum_{l=1}^{L_F} F_F^{(l-1)} F_F^{(l)}\right)
\end{aligned} \quad (32)$$

where C_{GC} and C_{FC} represent the computational complexity of graph convolutional layers and fully connected layers, respectively. L_G denotes the number of graph convolution layers, $|E^{(l)}|$ and $Q^{(l)}$ represents the number of edges and nodes in the l -th graph convolution respectively. F_G represents the input dimensions in the l -th graph convolution layer. $F_F^{(l-1)}$ and $F_F^{(l)}$ represent the input and output dimensions in the l -th fully connected layer. The computational complexity of SCM-GNN comes from the computation of convolution layers, graph convolution layers, SAGPool layers, and fully connected layers. The computational complexity of SCM-GNN can be calculated as

$$\begin{aligned}
C_{Cheb} &\sim O\left(\sum_{l=1}^{L_G+L_P} (K^2 |E^{(l)}| F_G^{(l)} + Q^{(l)} (F_G^{(l)})^2)\right) \\
C_{SCM-GNN} &\sim C_{Conv} + C_{Cheb} + C_{FC}
\end{aligned} \quad (33)$$

where C_{Conv} , C_{Cheb} and, C_{FC} represent the computational complexity of convolutional layers, graph convolutional layers, and fully connected layers, respectively. K is the order of Chebyshev approximation and L_P denotes the number of graph pooling layers.

The computational complexities of different algorithms are summarized in Table II. The N_t and N_e represent the number of samples and epochs in training, respectively. The computational complexity of model-based algorithms mainly is composed of the construction of test statistics and offline setting of thresholds. For data-driven algorithms, the main computational complexity arises from the calculation of neural

networks during offline training and online detection phases. Although the data-driven algorithms demonstrate superior detection accuracy, these algorithms suffer from limitations in computational complexity. Therefore, the model-based algorithms exhibit superior computational efficiency compared to data-driven algorithms. The SCM-GNN algorithm exhibits superiority over DS2MA, with a slight gap compared to GCN and CM-CNN. In terms of model parameters, although the SCM-GNN algorithm still remains a slight gap compared to GCN, it exhibits a notable advantage over both DS2MA and CM-CNN.

VI. CONCLUSION

In this work, we propose the SCM-GNN algorithm, which employs a graph convolution network and a covariance matrix with smooth factor, effectively adapting to complex and practical channel environments. The SCM-GNN algorithm performs multi-antenna spectrum sensing without requiring additional prior information from PUs, except for a certain amount of historical sensing data during model training. The extensive experiments have demonstrated that the SCM-GNN algorithm obtains superior detection performance compared to other benchmark methods in terms of detection performance, scalability, and robustness.

For future research, given the superiority of graph neural networks in addressing non-Euclidean distance and non-grid like data, the authors will investigate the feasibility and effectiveness of graph neural network-based cooperative spectrum sensing. Moreover, considering the lack of publicly effective datasets in the field of multi-antenna spectrum sensing, the authors will attempt to construct a dataset based on real-world scenarios.

REFERENCES

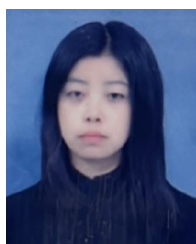
- [1] A. Gupta and R. K. Jha, "A survey of 5G network: Architecture and emerging technologies," *IEEE Access*, vol. 3, pp. 1206–1232, 2015

- [2] R. Li et al., "Intelligent 5G: When cellular networks meet artificial intelligence," *IEEE Wireless Commun.*, vol. 24, no. 5, pp. 175–183, Oct. 2017.
- [3] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A survey on enabling technologies, protocols, and applications," *IEEE Commun. Surveys Tuts.*, vol. 17, no. 4, pp. 2347–2376, 4th Quart., 2015.
- [4] J. Mitola and G. Q. Maguire, "Cognitive radio: Making software radios more personal," *IEEE Pers. Commun.*, vol. 6, no. 4, pp. 13–18, Aug. 1999.
- [5] L. Zhang, M. Xiao, G. Wu, M. Alam, Y.-C. Liang, and S. Li, "A survey of advanced techniques for spectrum sharing in 5G networks," *IEEE Wireless Commun.*, vol. 24, no. 5, pp. 44–51, Oct. 2017.
- [6] F. F. Digham, M.-S. Alouini, and M. K. Simon, "On the energy detection of unknown signals over fading channels," *IEEE Trans. Commun.*, vol. 55, no. 1, pp. 21–24, Jan. 2007.
- [7] Y. Zeng, C. L. Koh, and Y.-C. Liang, "Maximum eigenvalue detection: Theory and application," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Beijing, China, 2008, pp. 4160–4164.
- [8] R. Zhang, T. J. Lim, Y.-C. Liang, and Y. Zeng, "Multi-antenna based spectrum sensing for cognitive radios: A GLRT approach," *IEEE Trans. Commun.*, vol. 58, no. 1, pp. 84–88, Jan. 2010.
- [9] Y. Zeng, Y.-C. Liang, and R. Zhang, "Blindly combined energy detection for spectrum sensing in cognitive radio," *IEEE Signal Process. Lett.*, vol. 15, pp. 649–652, Oct. 2008.
- [10] Y. Zeng and Y.-C. Liang, "Spectrum-sensing algorithms for cognitive radio based on statistical covariances," *IEEE Trans. Veh. Technol.*, vol. 58, no. 4, pp. 1804–1815, May 2009.
- [11] Y. Zeng and Y.-C. Liang, "Eigenvalue-based spectrum sensing algorithms for cognitive radio," *IEEE Trans. Commun.*, vol. 57, no. 6, pp. 1784–1793, Jun. 2009.
- [12] W. Zhao, H. Li, M. Jin, Y. Liu, and S.-J. Yoo, "Eigenvalues-based universal spectrum sensing algorithm in cognitive radio networks," *IEEE Syst. J.*, vol. 15, no. 3, pp. 3391–3402, Sep. 2021.
- [13] M. Bkassiny, Y. Li, and S. K. Jayaweera, "A survey on machine-learning techniques in cognitive radios," *IEEE Commun. Surveys Tuts.*, vol. 15, no. 3, pp. 1136–1159, 3rd Quart., 2013.
- [14] Y. Cui, X. J. Jing, S. Sun, X. Wang, D. Cheng, and H. Huang, "Deep learning based primary user classification in cognitive radios," in *Proc. 15th Int. Symp. Commun. Inf. Technol. (ISCIT)*, 2015, pp. 165–168.
- [15] Y. Huang, H. Jiang, H. Hu, and Y. Yao, "Design of learning engine based on support vector machine in cognitive radio," in *Proc. Int. Conf. Comput. Intell. Softw. Eng.*, 2009, pp. 1–4.
- [16] M. M. Ramon, T. Atwood, S. Barbin, and C. G. Christodoulou, "Signal classification with an SVM-FFT approach for feature extraction in cognitive radio," in *Proc. SBMO/IEEE MTT-S Int. Microw. Optoelectron. Conf.*, 2009, pp. 286–289.
- [17] Y.-J. Tang, Q.-Y. Zhang, and W. Lin, "Artificial neural network based spectrum sensing method for cognitive radio," in *Proc. IEEE Int. Conf. Wireless Commun. Netw. Mobile Comput. (WiCOM)*, Chengdu, China, 2010, pp. 1–4.
- [18] Y. Xu, P. Cheng, Z. Chen, Y. Li, and B. Vucetic, "Mobile collaborative spectrum sensing for heterogeneous networks: A Bayesian machine learning approach," *IEEE Trans. Signal Process.*, vol. 66, no. 21, pp. 5634–5647, Nov. 2018.
- [19] Z. Shi, W. Gao, S. Zhang, J. Liu, and N. Kato, "Machine learning-enabled cooperative spectrum sensing for non-orthogonal multiple access," *IEEE Trans. Wireless Commun.*, vol. 19, no. 9, pp. 5692–5702, Sep. 2020.
- [20] C. Liu, J. Wang, and X. Liu, and Y.-C. Liang, "Deep CM-CNN for spectrum sensing in cognitive radio," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 10, pp. 2306–2321, Oct. 2019.
- [21] Z. Chen, Y.-Q. Xu, H. Wang, and D. Guo, "Deep STFT-CNN for spectrum sensing in cognitive radio," *IEEE Commun. Lett.*, vol. 25, no. 3, pp. 864–868, Mar. 2021.
- [22] P. Zhen, B. Zhang, Z. Chen, D. Guo, and W. Ma, "Spectrum sensing method based on wavelet transform and residual network," *IEEE Wireless Commun. Lett.*, vol. 11, no. 12, pp. 2517–2521, Dec. 2022.
- [23] Q. Cheng, Z. Shi, and J. Yuan, "Spectrum sensing in full-duplex OFDM systems using one-shot learning," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Montreal, QC, Canada, 2021, pp. 1–6.
- [24] A. Mehrabian, M. Sabbaghian, and H. Yanikomeroglu, "CNN-based detector for spectrum sensing with general noise models," *IEEE Trans. Wireless Commun.*, vol. 22, no. 2, pp. 1235–1249, Feb. 2023.
- [25] J. Xie, C. Liu, Y.-C. Liang, and J. Fang, "Activity pattern aware spectrum sensing: A CNN-based deep learning approach," *IEEE Commun. Lett.*, vol. 23, no. 6, pp. 1025–1028, Jun. 2019.
- [26] J. Xie, J. Fang, C. Liu, and X. Li, "Deep learning-based spectrum sensing in cognitive radio: A CNN-LSTM approach," *IEEE Commun. Lett.*, vol. 24, no. 10, pp. 2196–2200, Oct. 2020.
- [27] D. Ke, Z. Huang, X. Wang, and X. Li, "Blind detection techniques for non-cooperative communication signals based on deep learning," *IEEE Access*, vol. 7, pp. 89218–89225, 2019.
- [28] K. Chae and Y. Kim, "DS2MA: A deep learning-based spectrum sensing scheme for a multi-antenna receiver," *IEEE Wireless Commun. Lett.*, vol. 12, no. 6, pp. 952–956, Jun. 2023.
- [29] D. Janu, S. Kumar, and K. Singh, "A graph convolution network based adaptive cooperative spectrum sensing in cognitive radio network," *IEEE Trans. Veh. Technol.*, vol. 72, no. 2, pp. 2269–2279, Feb. 2023.
- [30] Y. Zhang et al., "SpecKriging: GNN-Based secure cooperative spectrum sensing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 11, pp. 9936–9946, Nov. 2022.
- [31] Y. S. Cho, J. Kim, W. Y. Yang, and C. G. Kang, "SISO channel models" in *MIMO-OFDM Wireless Communication with MATLAB*, New York, NY, USA: Wiley, 2010, ch. 2, sec. 2, pp. 38–47.
- [32] M. Nakagami, "The m -distribution—A general formula of intensity distribution of rapid fading," in *Statistical Methods in Radio Wave Propagation*, W. C. Hoffman, Eds., Oxford, U.K.: Elsevier, 1960, pp. 3–36.
- [33] R. A. A. Souza, R. Cogliatti, and M. D. Yacoub, "Efficient acceptance-rejection method for Nakagami- m complex samples," *IEEE Wireless Commun. Lett.*, vol. 3, no. 1, pp. 94–96, Feb. 2014.
- [34] M. Novey, T. Adali, and A. Roy, "A complex generalized gaussian distribution—Characterization, generation, and estimation," *IEEE Trans. Signal Process.*, vol. 58, no. 3, pp. 1427–1433, Mar. 2010.
- [35] W. Lee, M. Kim, and D.-H. Cho, "Deep cooperative sensing: Cooperative spectrum sensing based on convolutional neural networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 3, pp. 3005–3009, Mar. 2019.
- [36] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.
- [37] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," 2016, *arXiv:1606.09375*.
- [38] M. He, Z. Wei, and J.-R. Wen, "Convolutional neural networks on graphs with Chebyshev approximation, revisited," 2022, *arXiv:2202.03580*.
- [39] J. Lee, I. Lee, and J. Kang, "Self-attention graph pooling," 2019, *arXiv:1904.08082*.
- [40] H. Gao and S. Ji, "Graph U-nets," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 9, pp. 4948–4960, Sep. 2022.



Youqiang Dong received the B.E. degree in measurement and control technology and instrumentation from the Shandong University of Science and Technology, Qingdao, China, in 2020. He is currently pursuing the Ph.D. degree in instrument science and technology with the School of Aerospace Science and Technology, Xidian University, Xi'an, China.

His research interests include cognitive radio and graph neural networks.



Min Zhang (Member, IEEE) received the B.E. and Ph.D. degrees in measurement and control from Xidian University, Xi'an, China, in 2010 and 2017, respectively.

She is currently an Associate Professor with the School of Aerospace Science and Technology, Xidian University. Her research interests include time and frequency measurements, digital signal processing, object detection, and hyperspectral images processing.



Xi Cheng (Student Member, IEEE) received the M.S. degree in control theory and control engineering from the Zhengzhou University of Light Industry, Zhengzhou, China, in 2020. He is currently pursuing the Ph.D. degree in instrument science and technology with the School of Aerospace Science and Technology, Xidian University, Xi'an, China.

His research interests include hyperspectral anomaly detection and saliency object detection.



Hai Wang received the M.S. degree in communication and information system and the Ph.D. degree in instrument science and technology from Xidian University, Xi'an, China, in 2003 and 2007, respectively.

He is currently a Professor with the School of Aerospace Science and Technology, Xidian University. His research interests include the time and frequency measurements, frequency and phase measurements methods based on field-programmable gates array and system-on-chip, electronic instrument design, time-frequency signal processing, object detection, and hyperspectral images processing.